



Toward Quantum Competitiveness in Adiabatic Quantum Computing: A Comparative Study of Variational Algorithms

Philipp Hanussek

Master 2 Quantum Information, Sorbonne Université, Paris, France

Supervisor: dr hab. Bartłomiej Gardas, IITiS PAN, Gliwice, Poland

August 31, 2026

Abstract

We compare classical samplers, quantum annealers, and physics-inspired hardware within a single rigorous framework for quantum-assisted variational Monte Carlo. A restricted Boltzmann machine represents the wavefunction and is trained with stochastic reconfiguration. We evaluate the following classical and quantum samplers: Metropolis, Gibbs, simulated annealing, an FPGA-based solver, and D-Wave quantum annealers (Pegasus, Zephyr). We report on their behavior across different physical models: the transverse-field Ising model and the (frustrated) Heisenberg chain. We assess sampler performance by two metrics under a matched training protocol: time to a validated energy tolerance, and energy consumed to reach it. FPGA is the fastest and most energy-efficient of the metered classical solvers (we do not measure QPU energy consumption, so no energy comparison is made). On time-to-solution, Pegasus and Zephyr (with an effective-temperature correction applied) both scale nearly flat with N and overtake FPGA by the largest system size tested. However, given the small number of system sizes tested no claims regarding quantum advantage are made. We evaluate a native sparse embedding on the quantum annealer's hardware graph and find that it loses representational capacity. Embedding multiple problem copies in parallel is a promising direction for saving QPU time, though further experiments are needed for a solid claim. The variational ansatz used here is restricted to non-negative amplitudes. We experimentally apply the Marshall sign rule and find that it can extend the variational ansatz' validity to a moderately frustrated Heisenberg chain.

Contents

1	Overview	4
1.1	Previous work	4
2	Quantum Models	5
2.1	Quantum Ising Models	5
2.2	Heisenberg Models	5
2.3	Exact Solving	5
2.3.1	TFIM ground state solving in $O(N)$	5
2.3.2	Exact Diagonalization	6
3	Classical and quantum samplers	6
3.1	Adiabatic Quantum Computing	6
3.2	Metropolis-Hastings inspired methods	7
3.2.1	Metropolis-Hastings Algorithm	7
3.2.2	Simulated Annealing	7
3.3	Gibbs Sampling	7
4	Variational Algorithms	8
4.1	Restricted Boltzmann Machine	9
4.2	Training a RBM	9
4.3	Stochastic gradient descent versus Stochastic reconfiguration	10
4.3.1	Stochastic reconfiguration	10
4.3.2	Conjugate gradient	11
5	Experimental Analysis	11
5.1	Sampling Quality	11
5.2	Sampling temperature & CEM	12
5.2.1	D-Wave QPU	13
5.2.2	CEM	13
5.2.3	Simulated Annealing	14
5.3	Improving D-Wave Topology performance	14
5.3.1	Sparse embedding	14
5.3.2	Parallel Embeddings	16
5.4	Sign Problem in Quantum Monte Carlo	16
5.4.1	Marshall sign rule	17
5.5	Time-to- ϵ across solvers	18
5.5.1	Methodology	18
5.5.2	Results analysis	19
6	Conclusion	19
A	Supplementary Material	21
A.1	JAX	21
A.1.1	Just-in-time compilation	21
A.1.2	Vectorized Map	21
A.2	Hardware and Software Configuration	21
A.3	Convergence trajectories across models, sizes, and solvers	22

Glossary

CEM Conditional Expectation Matching. 13–15, 19, 20, 22

CG conjugate gradient. 11

JIT just-in-time. 19, 21

MCMC Markov chain Monte Carlo. 7, 11, 19, 22

QUBO quadratic unconstrained binary optimization. 4

RBM restricted Boltzmann machine. 8–18

SA simulated annealing. 7, 12, 15, 19, 22

SAPI Solver API. 19

SR stochastic reconfiguration. 8, 10, 11, 13, 18, 19, 21, 22

TFIM Transverse Field Ising Model. 5, 9, 12, 13, 15–18, 22, 23

TTE time to ϵ . 19, 20

TTS time to solution. 4, 19

VQE Variational Quantum Eigensolver. 8

1 Overview

Since the development of programmable quantum processors, many claims regarding quantum advantage have been made. Often, these advantage claims did not last long. In this work, we focus exclusively on the D-Wave Quantum Annealer, which leverages quantum technology to solve quadratic unconstrained binary optimization (QUBO) problems. D-Wave is among the first companies to commercially sell quantum computers. In 2025, researchers from D-Wave claimed to have solved a scientifically relevant problem faster using quantum technology than it would have been possible on classical devices [1]. However, it is currently an open question whether the D-Wave Quantum Annealer is able to solve other problems well, and possibly faster than conventional computers. In this work, we assess D-Wave’s ability to sample from a probability distribution, to approximate the ground state of different statistical mechanical models using variational algorithms.

1.1 Previous work

The D-Wave Quantum Annealer’s performance scaling for finding the ground state (corresponding to the minimum of the quadratic binary cost function) has been extensively studied, in the area of cryptography and physical simulation. In previous work, we found that classical methods exhibit faster solving times and better scaling (see Figure 1).

We briefly report on previous work in the field of simulating physical systems using quantum annealing hardware. In [2], we used a method presented in [3] to encode the solution to the wave function of different Hamiltonians into a QUBO problem. These instances were then solved on different physical and physics-inspired solvers. We evaluate the scaling of the samplers using the time to solution (TTS) metric, which corresponds to the computational time needed to solve the optimisation problem with a success probability of at least P_{target} . The TTS is defined as

$$\text{TTS} = \frac{\ln(1 - P_{\text{target}})}{\ln(1 - P_{\text{success}})} T_r, \quad (1.1)$$

where P_{success} is the probability of obtaining the ground state in a single run, and T_r denotes the runtime of a single run. Figure 1a shows the resulting TTS for the native, embedding-free formulation of one such Hamiltonian; classical solvers dominate across the entire range tested ($N = 16$ to 112).

Another previous work that we report on is integer factorisation on the D-Wave Annealer, that is, computing the two prime factors p, q of a semiprime $N = p \times q$. [4] benchmark three QUBO adaptations of this problem against uniformly random guessing. Figure 1b reproduces their result: all three encodings still scale exponentially in the number of unknown bits $l = l_p^* + l_q^*$, and only the instances using custom embedding (denoted as CFA[5]), asymptotically beats random guessing.

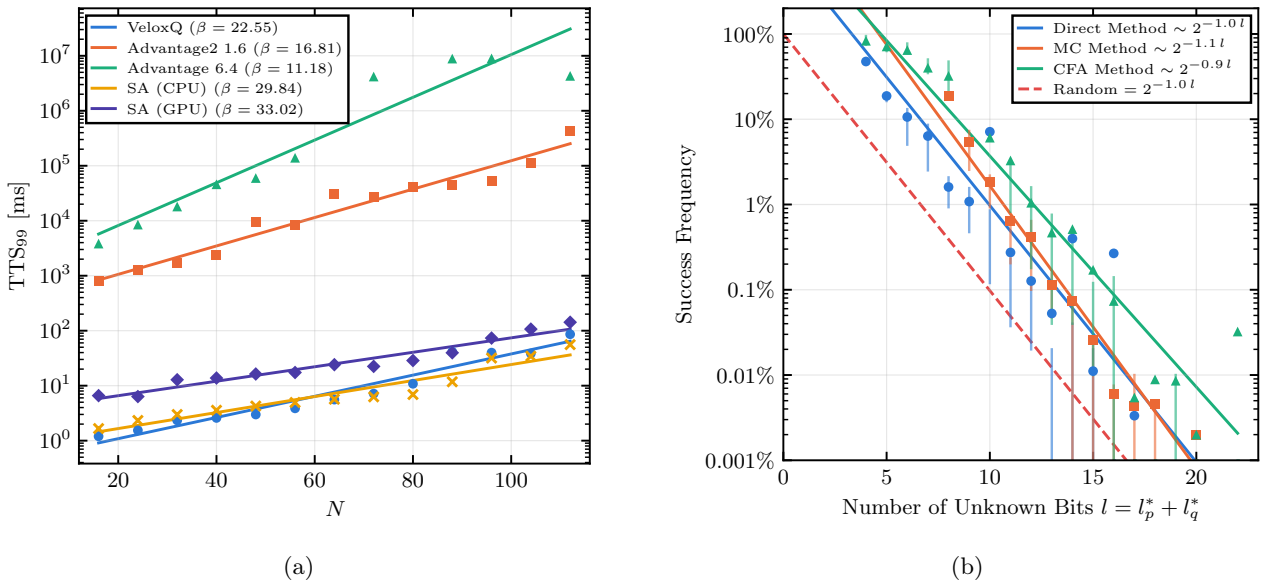


Figure 1: Illustration of previous results on solving optimization problems using QPU hardware. (a) TTS₉₉ for simulating a native single-qubit rotation, governed by $H_1 = \frac{\pi}{2}\sigma_y$, vs. system size N [2], missing points indicate zero success probability. (b) Success frequency of factoring a semiprime $N = p \times q$ vs. the number of unknown bits $l = l_p^* + l_q^*$, reproduced from [4], for three different QUBO encodings and for uniformly random guessing; markers are the mean success frequency over ~ 10 random semiprimes per l , bars span the 25th–75th percentile, and lines are fitted exponential scalings $\propto 2^{-bl}$.

To conclude, results returned by the quantum annealer are inherently noisy. This makes it difficult to sample the *exact* ground state of the submitted Hamiltonian. In this work, we therefore assess the device’s ability to act as a Gibbs sampler and sample from the Boltzmann distribution. It has been shown earlier that the D-Wave device can in fact be used as a Gibbs sampler[6, 7] and that these samples can be used to train small-scale neural networks to solve many-body problems[8]. In this work we aim to extend the analysis to different many-body models, notably the J_1 - J_2 Heisenberg model, and propose a rigorous comparison between different quantum and classical samplers for bigger system sizes.

2 Quantum Models

2.1 Quantum Ising Models

The Ising model consists of discrete spins arranged on a lattice, interacting via nearest-neighbour couplings. In its quantum extension, the Transverse Field Ising Model (TFIM), a transverse magnetic field is added, introducing quantum fluctuations through non-commuting terms in the Hamiltonian [9]. The Hamiltonian for the one-dimensional TFIM is given by

$$H = -J \sum_i \sigma_i^z \sigma_{i+1}^z - h \sum_i \sigma_i^x \quad (2.1)$$

where $J > 0$ is the ferromagnetic coupling strength, $h \geq 0$ is the transverse magnetic field strength, and periodic boundary conditions are imposed on the chain. The 1D TFIM serves as a benchmark for our models, as it is exactly solvable via the Jordan–Wigner transformation. Despite its simplicity, it displays non-trivial quantum behavior, including a quantum phase transition.

2.2 Heisenberg Models

The Heisenberg spin chain describes spins on lattice sites interacting via exchange coupling [10]:

$$H = \sum_{\langle i,j \rangle} J_{ij} \boldsymbol{\sigma}_i \cdot \boldsymbol{\sigma}_j \quad (2.2)$$

where $\boldsymbol{\sigma}_i = (\sigma_i^x, \sigma_i^y, \sigma_i^z)$ are the Pauli spin operators, J_{ij} is the exchange coupling between sites i and j , and $\langle i, j \rangle$ denotes nearest-neighbour pairs. Spin operators are represented by Pauli matrices with eigenvalues ± 1 . In contrast to the TFIM, the Heisenberg model couples all three spin components.

The frustrated J_1 - J_2 Heisenberg chain extends the Heisenberg model with a next-nearest-neighbour exchange term:

$$H = J_1 \sum_i \boldsymbol{\sigma}_i \cdot \boldsymbol{\sigma}_{i+1} + J_2 \sum_i \boldsymbol{\sigma}_i \cdot \boldsymbol{\sigma}_{i+2}, \quad (2.3)$$

where J_1 and J_2 are the nearest- and next-nearest-neighbour coupling strengths. The competing interactions frustrate the system, preventing simultaneous minimisation of both exchange terms. Setting $J_2 = 0$ recovers the standard Heisenberg chain, while increasing J_2/J_1 drives the system into a more frustrated regime that poses a more stringent test for variational methods.

2.3 Exact Solving

Determining the exact value of the ground state up to numerical precision is crucial for evaluating the performance of the trained models. For TFIM, exact solving is possible for arbitrary system sizes, whereas solving the J_1 - J_2 Heisenberg model is computationally hard.

2.3.1 TFIM ground state solving in $O(N)$

The 1D TFIM defined in Equation 2.1 can be solved exactly in $O(N)$ time, for $J > 0$, $h \geq 0$ with periodic boundary condition. We closely follow the method presented in [11][12], to exactly solve the model, by mapping the spins to fermions using the Jordan–Wigner transformation, and thereby splitting it into a periodic and an antiperiodic boundary-condition sector for the fermions. In accordance with [12], we find that only the fermionic sector with antiperiodic boundary condition is relevant for analytically computing the ground state. Therefore, using the dispersion relation

$$\omega(k) = \sqrt{J^2 + h^2 - 2Jh \cos(k)} \quad (2.4)$$

, the closed form for the ground state is given by

$$E_0 = - \sum_{m=0}^{N-1} \omega \left(\frac{\pi(2m+1)}{N} \right), \quad (2.5)$$

. We verified this closed-form result numerically against exact diagonalization (**netket**, using **scipy**'s sparse **eigsh**) for $N \in \{4, 6, 8, 10\}$ and $h \in \{0.5, 1.0, 1.5\}$, finding agreement to machine precision ($< 10^{-13}$).

2.3.2 Exact Diagonalization

In cases where mapping to a quadratic fermion Hamiltonian is not possible, e.g. for the frustrated J_1 - J_2 Heisenberg chain, exact diagonalization is used for small system sizes N . Exact diagonalization works by enumerating all 2^N basis states of the Hilbert space, constructing the Hamiltonian as a $2^N \times 2^N$ matrix in that basis, and finding its lowest eigenvalue via the Lanczos algorithm. The Lanczos algorithm avoids diagonalizing the entire matrix and only returns the matrix's lowest eigenvalue. We use **scipy**'s **eigsh** implementation of this algorithm.

3 Classical and quantum samplers

3.1 Adiabatic Quantum Computing

Adiabatic Quantum Computing is based on the adiabatic theorem [13, 14], which states: *A physical system remains in its instantaneous eigenstate if a given perturbation is acting on it slowly enough and if there is a gap between the eigenvalue and the rest of the Hamiltonian's spectrum.* In this work we focus on the D-Wave Quantum Annealer, which acts according to the adiabatic theorem in the following way: In the beginning of the annealing process, the device is initiated in the ground state of an initial driver Hamiltonian H_D that can be easily constructed:

$$H_D = - \sum_{i=1}^N \sigma_x^{(i)} \text{ with ground state } |+\rangle^{\otimes N} \quad (3.1)$$

where σ_x and σ_z are the Pauli matrices acting on qubit i . The problem Hamiltonian is defined as

$$H_P = \sum_{i=1}^N h_i \sigma_z^{(i)} + \sum_{i=1}^N \sum_{j=1}^{i-1} J_{ij} \sigma_z^{(i)} \otimes \sigma_z^{(j)}. \quad (3.2)$$

During the annealing process, the system is slowly brought from H_D to H_P according to the so-called annealing schedules $A(s)$ and $B(s)$:

$$H(s) = A(s)H_D + B(s)H_P = -A(s) \sum_{i=1}^N \sigma_x^{(i)} + B(s) \sum_{i=1}^N h_i \sigma_z^{(i)} + B(s) \sum_{i=1}^N \sum_{j=1}^{i-1} J_{ij} \sigma_z^{(i)} \otimes \sigma_z^{(j)}, \quad (3.3)$$

where $s \in [0, 1]$ is the normalized time. $A(s)$ decreases the driving terms in H_D as s increases, and $B(s)$ activates the terms of the problem Hamiltonian H_P . At the end of the evolution, if the process happens slow enough, and assuming a noiseless environment, the system will be in the ground state of the problem Hamiltonian according to the adiabatic theorem. It is important to note that the qubits in the D-Wave Quantum Annealer are not *all-to-all* connected. The qubits are arranged in an architecture dependent grid. In the Pegasus topology, one qubit couples on average to 15 other qubits, while Zephyr qubits couple on average to 20 other qubits. Therefore, most problems require an additional embedding step, in which multiple physical qubits are *chained* together to form one logical qubit. For a toy-problem example of embedding a triangle shaped connectivity pattern on a QPU, see Figure 2c. As all problems examined in this work can be represented as bipartite graphs, we use **minorminor.busclique**, which is part of D-Wave's Ocean SDK. There exist three different types of couplers, connecting physical qubits:

- **Internal Couplers** connecting qubits of opposite orientation
- **External Couplers** connecting qubits aligned in parallel
- **Odd Couplers** which connect similarly aligned pairs of qubits

See Figure 3 for visualization of different QPU topologies.

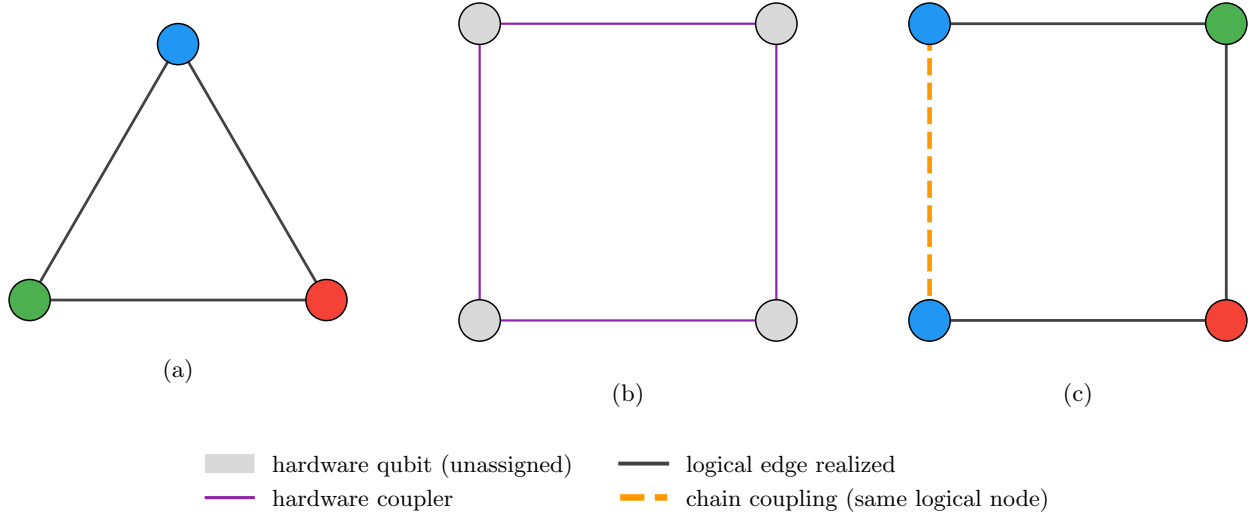


Figure 2: Minor embedding of a triangle onto a sparser hardware graph. **(a)** Logical problem graph: a triangle requires a coupler between every pair of nodes. **(b)** Target hardware graph: a 4-cycle, whose qubits admit no triangle directly. **(c)** A valid embedding: one logical node is realized as a chain of two physical qubits (joined by a strong ferromagnetic coupling, dashed), so that the three logical edges are each realized by a single physical coupler.

3.2 Metropolis-Hastings inspired methods

We use several Markov chain Monte Carlo (MCMC) methods to generate samples from probability distribution from which direct sampling would be difficult.

3.2.1 Metropolis-Hastings Algorithm

The Metropolis-Hastings Algorithm [15, 16] is a MCMC method used to obtain samples from a (multivariate) probability distribution $P(x)$ from which sampling is difficult. The goal of the algorithm is to define a Markov chain whose stationary distribution is $P(x)$. Suppose we can evaluate an unnormalized target density $f(x)$, where $f(x) \propto P(x)$, and the normalization constant is unknown. In the **proposal** step, a *candidate* x^* from a proposal distribution $q(x^*|x_n)$ is generated, x_n being the current state of the Markov chain. In the **accept-reject** step, the acceptance probability $A(x_n \rightarrow x^*)$ is calculated [15, 16]:

$$A(x_n \rightarrow x^*) = \min \left(1, \frac{f(x^*)}{f(x_n)} \times \frac{q(x_n|x^*)}{q(x^*|x_n)} \right). \quad (3.4)$$

For the evaluation of $\frac{f(x^*)}{f(x_n)}$, knowledge of the normalization constant is not necessary. The candidate x^* is accepted with probability $A(x_n \rightarrow x^*)$.

3.2.2 Simulated Annealing

Simulated annealing (SA) [17] is the classical analogue and predecessor of quantum annealing. It works by exploring the neighborhood of a given configuration, accepting the proposed solution with a temperature dependent probability, or if the new configuration has a lower energy than the current one. In this work, we use a geometrical temperature schedule [18] instead of a linear one. The schedule is given by the formula

$$T_k = T_{\text{initial}} \left(\frac{T_{\text{final}}}{T_{\text{initial}}} \right)^{k/(n_{\text{steps}}-1)}, \quad k = 0, 1, \dots, n_{\text{steps}} - 1, \quad (3.5)$$

where T_{initial} and T_{final} are the starting and target temperatures and n_{steps} is the total number of annealing steps.

3.3 Gibbs Sampling

Gibbs sampling [19] is used when sampling directly from a joint distribution is difficult, but sampling from the full conditional distributions is tractable. The algorithm constructs a Markov chain by iteratively sampling each

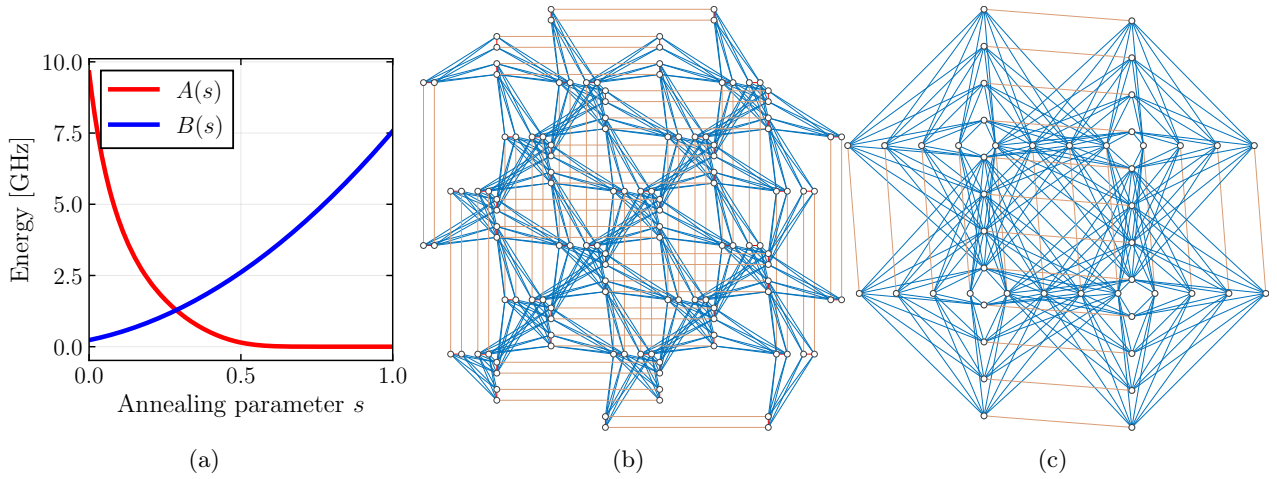


Figure 3: (a) Time-dependent energy scales executed by the D-Wave quantum annealer. The coefficient $A(s)$ (red) multiplies the transverse-field term that enables quantum tunnelling and therefore falls rapidly as the run progresses. The coefficient $B(s)$ (blue) multiplies the problem Hamiltonian and rises smoothly, so that the device moves from a tunnelling-dominated regime at $s = 0$ to a problem-dominated regime at $s = 1$. (b) Native connectivity map for the Pegasus architecture (D-Wave Advantage). Circles mark individual superconducting qubits; blue lines are internal couplers and orange lines mark external couplers (c) Connectivity map for the newer Zephyr architecture (D-Wave Advantage2), which increases the number of couplers per qubit and introduces longer-range links.

variable (or block of variables) from its distribution conditioned on the current values of all other variables. This procedure generates a chain whose stationary distribution is the target joint distribution. In this work we use an adapted version of this procedure, *blocked* Gibbs sampling, which allows to sample from all the hidden (visible) variables together. We closely follow the method description given in D’Angelo et al.[20]. As we are using a restricted Boltzmann machine (RBM) which by definition is bipartite without any intra-layer connections, there is mutual independence between the hidden (visible) variables if the visible (hidden) variables are held fixed. The blocked Gibbs sampling can then be described by the following equations:

$$p(h_j = +1 \mid \mathbf{v}) = \sigma \left(2 \left(b_j + \sum_i W_{ij} v_i \right) \right), \quad p(v_i = +1 \mid \mathbf{h}) = \sigma \left(2 \left(\sum_j W_{ij} h_j - a_i \right) \right), \quad (3.6)$$

with $\sigma(x) = 1/(1+e^{-x})$, $\mathbf{a}$ the visible bias, $\mathbf{b}$ the hidden bias, and W the weight matrix. Since $h_j, v_i \in \{-1, +1\}$, each unit is sampled by drawing $u \sim \mathcal{U}(0, 1)$ and setting the unit to $+1$ if $u < p(\cdot = +1 \mid \cdot)$, else -1 . One full block-Gibbs sweep is

$$\mathbf{h} \sim p(\mathbf{h} \mid \mathbf{v}), \quad \mathbf{v}' \sim p(\mathbf{v} \mid \mathbf{h}), \quad (3.7)$$

with all components of $\mathbf{h}$ (resp. $\mathbf{v}'$) drawn simultaneously. Before sampling for the first time, we initialize the sampler by performing a so-called *warm-up*, that is, performing 200 full block-Gibbs sweeps to allow the chain to converge to its stationary distribution (in the optimal case). Under the *persistent chain* assumption from [20], no warm-up is performed for subsequent sampling calls. The PCD assumption states that the chain stays near-equilibrium for small parameter changes.

4 Variational Algorithms

Variational algorithms are a class of hybrid quantum-classical optimization methods designed to find the ground state of a given Hamiltonian. Unlike the Variational Quantum Eigensolver (VQE), which prepares and measures a parametrised circuit on a gate-based quantum computer, this work uses quantum-assisted Variational Monte Carlo: a classical neural-network ansatz [21] is optimised via stochastic reconfiguration (SR), and a quantum device is used only to generate samples of that ansatz. Variational Monte Carlo consists of three parts: the *cost function* to be minimized, the *ansatz*, which in our case is a Boltzmann machine, and an *expectation value estimation*. Additionally, a classical optimizer is needed to improve the ansatz at each iteration. Depending on how the ansatz is realised on quantum hardware, two approaches for estimating the expectation value are possible:

- On gate-based quantum computers, measuring can be used to gather expectation values of the modified ansatz and improve it further.

- In adiabatic quantum computing, the quantum device is used to generate representative samples of the modified ansatz. These samples are then used to approximate the expectation value.

As we are interested in comparing current quantum annealing devices with their classical counterparts, we focus solely on the second case.

4.1 Restricted Boltzmann Machine

Different constructions for the ansatz are possible, here we focus on a RBM which has been shown to successfully approach the ground state in previous works[20, 21, 8]. A RBM is an artificial neural network consisting of visible and hidden units that form a bipartite graph, without intra-layer connections. It has been shown to be a universal approximator, meaning that it can represent any desired probability distribution to arbitrary accuracy[22], given a sufficient number of hidden units. The main idea is that expectation values with respect to the quantum state can be estimated using samples drawn from the probability distribution $|\Psi(\mathbf{v})|^2$. Since the wavefunction ansatz is represented by a Restricted Boltzmann Machine, which corresponds to a classical stochastic Ising model, configurations $\mathbf{v}$ can be sampled approximately using a D-Wave quantum annealer. Commonly for RBMs, the ansatz is defined via an energy function over visible and hidden units,

$$E_{\theta}(\mathbf{v}, \mathbf{h}) = \mathbf{a} \cdot \mathbf{v} - \mathbf{b} \cdot \mathbf{h} - \mathbf{h} \cdot \mathbf{W}^{\top} \cdot \mathbf{v}, \quad (4.1)$$

with weights $\theta = \{\mathbf{a}, \mathbf{b}, \mathbf{W}\}$ respectively applied to the vector of visible units $\mathbf{v} = (v_1, \dots, v_N) \in \{-1, +1\}^N$ and hidden units $\mathbf{h} = (h_1, \dots, h_M) \in \{-1, +1\}^M$. Throughout, $\mathbf{W} \in \mathbb{R}^{N \times M}$ is indexed visible-by-hidden, W_{ij} with i the visible and j the hidden index, matching its representation in the implementation. The joint quantum state is

$$|\Psi\rangle = \sum_{\mathbf{v}} \Psi(\mathbf{v}) |\mathbf{v}\rangle, \quad \Psi(\mathbf{v}) = \sqrt{\sum_{\mathbf{h}} e^{-E_{\theta}(\mathbf{v}, \mathbf{h})}}, \quad (4.2)$$

so that $|\Psi(\mathbf{v})|^2 = \sum_{\mathbf{h}} e^{-E_{\theta}(\mathbf{v}, \mathbf{h})}$ is, up to normalization, the marginal over hidden units of the joint Boltzmann distribution $p_{\theta}(\mathbf{v}, \mathbf{h}) = Z^{-1} e^{-E_{\theta}(\mathbf{v}, \mathbf{h})}$, $Z = \sum_{\mathbf{v}, \mathbf{h}} e^{-E_{\theta}(\mathbf{v}, \mathbf{h})}$, i.e. $|\Psi(\mathbf{v})|^2 \propto \sum_{\mathbf{h}} p_{\theta}(\mathbf{v}, \mathbf{h})$. The corresponding Born probability distribution is

$$\rho(\mathbf{v}) = \frac{|\Psi(\mathbf{v})|^2}{\sum_{\mathbf{v}'} |\Psi(\mathbf{v}')|^2}, \quad (4.3)$$

which is however intractable to normalize for large Hilbert spaces. Tracing out the hidden variables in Equation 4.2, the ansatz becomes[21]:

$$\Psi(\mathbf{v}) = e^{-\mathbf{a} \cdot \mathbf{v}/2} \prod_{j=1}^{N_h} \left[2 \cosh \left(b_j + \sum_i W_{ij} v_i \right) \right]^{1/2}. \quad (4.4)$$

Gibbs sampling on the D-Wave annealer instead targets the joint distribution $p_{\theta}(\mathbf{v}, \mathbf{h})$ where both $\mathbf{v}$ and $\mathbf{h}$ are encoded as physical qubits (Figure 4). The annealer returns samples of the pair $(\mathbf{v}, \mathbf{h})$. $\mathbf{h}$ is subsequently marginalized to obtain samples of $\mathbf{v}$ distributed as $\rho(\mathbf{v})$.

4.2 Training a RBM

The RBM training loop can be summarized in four steps:

1. Sampling a configuration of visible variables $\mathbf{v}$ using classical / QPU samplers (see previous section)
2. Calculating the **local energies**: The local energy is given by[8]

$$E_{\text{loc}}(\mathbf{v}) = \frac{\langle \mathbf{v} | H | \Psi \rangle}{\Psi(\mathbf{v})} \quad (4.5)$$

(see Equation 4.11 for the expanded form used during training). For the TFIM (2.1), evaluating this only requires the ansatz (4.4) at the current configuration and at each single-spin-flipped configuration $\bar{\mathbf{v}}_i$ (i.e. $\mathbf{v}$ with $v_i \rightarrow -v_i$)[8, 21]:

$$E_{\text{loc}}(\mathbf{v}) = -J \sum_{\langle i, j \rangle} v_i v_j - h \sum_i \frac{\Psi(\bar{\mathbf{v}}_i)}{\Psi(\mathbf{v})}. \quad (4.6)$$

For the Heisenberg and frustrated J_1 - J_2 chains (2.3), the off-diagonal part works differently[21, 23]. Instead of a field flipping one spin at a time, the $\sigma_i^x \sigma_j^x + \sigma_i^y \sigma_j^y$ exchange term flips a *pair* of spins (i, j) together, and only when they point in opposite directions ($v_i \neq v_j$): flipping two equal spins would just reproduce

$\mathbf{v}$ itself, so that term contributes nothing. Writing $\bar{\mathbf{v}}_{ij}$ for $\mathbf{v}$ with both v_i and v_j flipped, and $s \in \{-1, +1\}$ for the Marshall sign (subsubsection 5.4.1, applied only to nearest-neighbour bonds), the local energy is

$$E_{\text{loc}}(\mathbf{v}) = J_1 \sum_i v_i v_{i+1} + J_2 \sum_i v_i v_{i+2} + 2J_1 s \sum_{i: v_i \neq v_{i+1}} \frac{\Psi(\bar{\mathbf{v}}_{i,i+1})}{\Psi(\mathbf{v})} + 2J_2 \sum_{i: v_i \neq v_{i+2}} \frac{\Psi(\bar{\mathbf{v}}_{i,i+2})}{\Psi(\mathbf{v})}. \quad (4.7)$$

The local energy formulation given here is an extension of the version given in Carleo et al.[21], as it includes the additional terms to take into account the next-nearest-neighbour J_2 coupling[23]. In this work, the RBM is restricted to be real-valued (in contrast to the complex-valued RBM used in [21]). Consequently, we need to apply a sign correction s to the J_1 term.

3. Calculating the **log-derivatives**[8]: Since the ansatz (4.4) already has the hidden units traced out, the gradient observable O_k needed for SR (subsection 4.3) can likewise be evaluated from the visible units alone. Writing $\theta_j = b_j + \sum_i W_{ij} v_i$,

$$\frac{\partial \log \Psi(\mathbf{v})}{\partial \theta_k} = \frac{1}{2} \begin{cases} -v_i, & \theta_k = a_i \\ \tanh(\theta_j), & \theta_k = b_j \\ v_i \tanh(\theta_j), & \theta_k = W_{ij}. \end{cases} \quad (4.8)$$

4. Updating the **parameters** $\theta = \{\mathbf{a}, \mathbf{b}, \mathbf{W}\}$ via SR, as detailed in subsection 4.3.

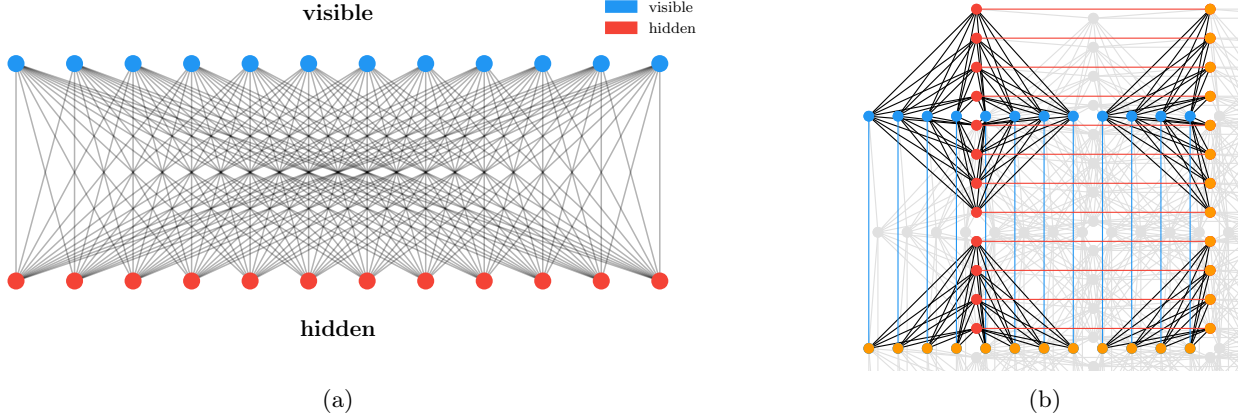


Figure 4: (a) RBM with 12 visible and 12 hidden units. (b) Embedding of the corresponding bipartite graph onto the Zephyr hardware topology. Blue nodes are visible units, red nodes are hidden units, orange nodes are auxiliary chain qubits. Each physical node is connected to one auxiliary qubit.

4.3 Stochastic gradient descent versus Stochastic reconfiguration

Traditionally, neural networks are trained using gradient descent. We will not go into details about this well known approach, as it is extensively covered by literature. Gradients are computed efficiently using backpropagation which exploits the chain rule, reusing intermediate derivatives to evaluate the compositions of continuous functions. Carleo et al., in their article on using neural networks for solving many-body problems ([21]) instead use a method known as SR, first described by Sorello et al. in [23, 24]. The main advantage of SR over gradient descent is that it exploits more knowledge about the ansatz than gradient descent. In particular, gradient descent lives in the *parameter space*, and does not take into account the influence of parameter changes on the actual wave function *probability space* $|\Psi|^2$.

4.3.1 Stochastic reconfiguration

SR is implemented as follows: given the logarithmic derivative for each parameter k , that is the gradient observable

$$O_k(\mathbf{v}) = \frac{\partial \log \Psi(\mathbf{v})}{\partial \theta_k}, \quad (4.9)$$

with $\theta = \{\mathbf{a}, \mathbf{b}, \mathbf{W}\}$, the energy gradient or force vector is defined as

$$F_k = \langle O_k E_{\text{loc}} \rangle - \langle O_k \rangle \langle E_{\text{loc}} \rangle, \quad (4.10)$$

which is the covariance of the gradient observable and the local energy. The local energy $E_{\text{loc}}(\mathbf{v})$ estimates the variational energy $\langle H \rangle$ of the current RBM ansatz using samples drawn from $|\Psi(\mathbf{v})|^2$ [21]:

$$E_{\text{loc}}(\mathbf{v}) = \frac{\langle \mathbf{v} | H | \Psi \rangle}{\langle \mathbf{v} | \Psi \rangle} = \sum_{\mathbf{v}'} H_{\mathbf{v}\mathbf{v}'} \frac{\Psi(\mathbf{v}')}{\Psi(\mathbf{v})} \quad (4.11)$$

The next step then is to define the quantum geometric tensor $\mathbf{S}$, the regularised sample covariance of the O_k vectors, which measures how sensitive the Born distribution $|\Psi|^2$ is to each parameter direction. In matrix form, if $\mathbf{O}$ is the matrix of mean-centred gradients, then $\mathbf{S} = \frac{1}{N_s} \mathbf{O}^\top \mathbf{O} + \lambda I$, with a small regularisation λ added for numerical stability. After solving the SR system

$$\mathbf{S} \cdot \delta \boldsymbol{\theta} = \mathbf{F} \quad (4.12)$$

for $\delta \boldsymbol{\theta}$, the parameters are updated by

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \delta \boldsymbol{\theta}, \quad (4.13)$$

with learning rate η .

Algorithm 1 SR

Require: Parameters $\boldsymbol{\theta} = \{\mathbf{a}, \mathbf{b}, \mathbf{W}\}$, learning rate η , regularisation λ

- 1: **for** $t = 1, 2, \dots$ **do**
 - 2: Sample $\mathbf{v}^{(s)} \sim |\Psi_{\boldsymbol{\theta}}|^2$ for $s = 1, \dots, N_s$ ▷ MCMC
 - 3: $E_{\text{loc}}(\mathbf{v}^{(s)}) \leftarrow \sum_{\mathbf{v}'} H_{\mathbf{v}\mathbf{v}'} \Psi(\mathbf{v}') / \Psi(\mathbf{v})$ for each s
 - 4: $O_k(\mathbf{v}^{(s)}) \leftarrow \partial \log \Psi(\mathbf{v}^{(s)}) / \partial \theta_k$ for each s, k
 - 5: $F_k \leftarrow \langle O_k E_{\text{loc}} \rangle - \langle O_k \rangle \langle E_{\text{loc}} \rangle$
 - 6: $\mathbf{S} \leftarrow \frac{1}{N_s} \mathbf{O}^\top \mathbf{O} + \lambda I$
 - 7: Solve $\mathbf{S} \delta \boldsymbol{\theta} = \mathbf{F}$ ▷ conjugate gradient, matrix-free
 - 8: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \delta \boldsymbol{\theta}$
 - 9: **end for**
-

4.3.2 Conjugate gradient

As $\mathbf{S} \in \mathbb{R}^{N_{\text{params}} \times N_{\text{params}}}$ grows quadratically with the number of parameters $N_{\text{params}} = N + M + NM$, with N and M the number of visible and hidden units respectively, solving the SR system in Equation 4.12 is intractable for large system sizes. We therefore apply a technique called conjugate gradient (CG), introduced by Hestenes et al. in [25], which requires only matrix vector products $\mathbf{S} \cdot \mathbf{x}$ and converges for any symmetric positive-definite matrix. Computing the matrix-vector product then costs only $\mathcal{O}(N_s \cdot N_{\text{params}})$:

$$\mathbf{S} \cdot \mathbf{x} = \frac{1}{N_s} \bar{\mathbf{O}}^\top (\bar{\mathbf{O}} \mathbf{x}) + \lambda \mathbf{x} \quad (4.14)$$

Both this matrix-free CG solve and the local-energy/gradient evaluations above are implemented in JAX for GPU acceleration; see subsection A.1 for implementation details.

5 Experimental Analysis

We conduct experiments analyzing sampler quality and physical properties of samplers and models.

5.1 Sampling Quality

While some of the samplers are guaranteed to sample from the Boltzmann distribution, others are not. In this section, we analyze the quality of samplers with regard to various parameters. One measure for sampling quality is the *total variation distance* D_{TV} . It is defined as

$$D_{\text{TV}}(\mu, \nu) = \frac{1}{2} \sum_{\boldsymbol{\sigma} \in \{-1, +1\}^N} |\mu(\boldsymbol{\sigma}) - \nu(\boldsymbol{\sigma})|, \quad (5.1)$$

where μ is the histogram returned from the sampler and $\nu(\boldsymbol{\sigma}) \propto |\Psi(\boldsymbol{\sigma})|^2$ is the RBM distribution. A related measure is the *Kullback-Leibler divergence* D_{KL} , defined as

$$D_{\text{KL}}(\mu \| \nu) = \sum_{\boldsymbol{\sigma} \in \{-1, +1\}^N} \mu(\boldsymbol{\sigma}) \log \frac{\mu(\boldsymbol{\sigma})}{\nu(\boldsymbol{\sigma})}. \quad (5.2)$$

D_{KL} is mainly used in this work as an objective function to be minimized when *fitting* a continuous parameter. As every sampler has a range of parameters, we are interested in how well the sampling performs with regards to the different tuning parameters. SA, Gibbs, and Metropolis can be tuned by the number of *sweeps*, that is, the number of single-spin-flip proposals.

To determine D_{TV} , we use a SA pretrained RBM and compute its exact distribution $|\Psi|^2$ numerically by enumerating all possible spin configurations. We then collect samples from different samplers and compare the resulting empirical histogram to the exact distribution.

The results are presented in Figure 5. As expected, it can be seen that Metropolis and Gibbs sampling perform theoretically near-optimal. The performance of SA improves as the transverse field bias h increases. In Figure 5(b) the exact histogram of configurations $|\Psi(\mathbf{v})|^2$ is plotted. For $h = 0.5$ the distribution is peaked around some highly probable sample configurations, whereas the distribution becomes more uniform for stronger values of h . A possible explanation for the performance increase of the SA sampler with h is that it could be easier to sample from a uniform distribution than from a peaked one.

For low h values, the spins in the TFIM are aligned (so-called ferromagnetic phase). With $h \rightarrow \infty$ the transverse field dominates and the spins tend to align in the x -direction, as the ground state becomes a product of single-spin σ_x eigenstates. Measured in the z -basis, this leads to an exactly uniform distribution over spin configurations for any N . This phase transition from the ferromagnetic to the paramagnetic phase is illustrated in Figure 6.

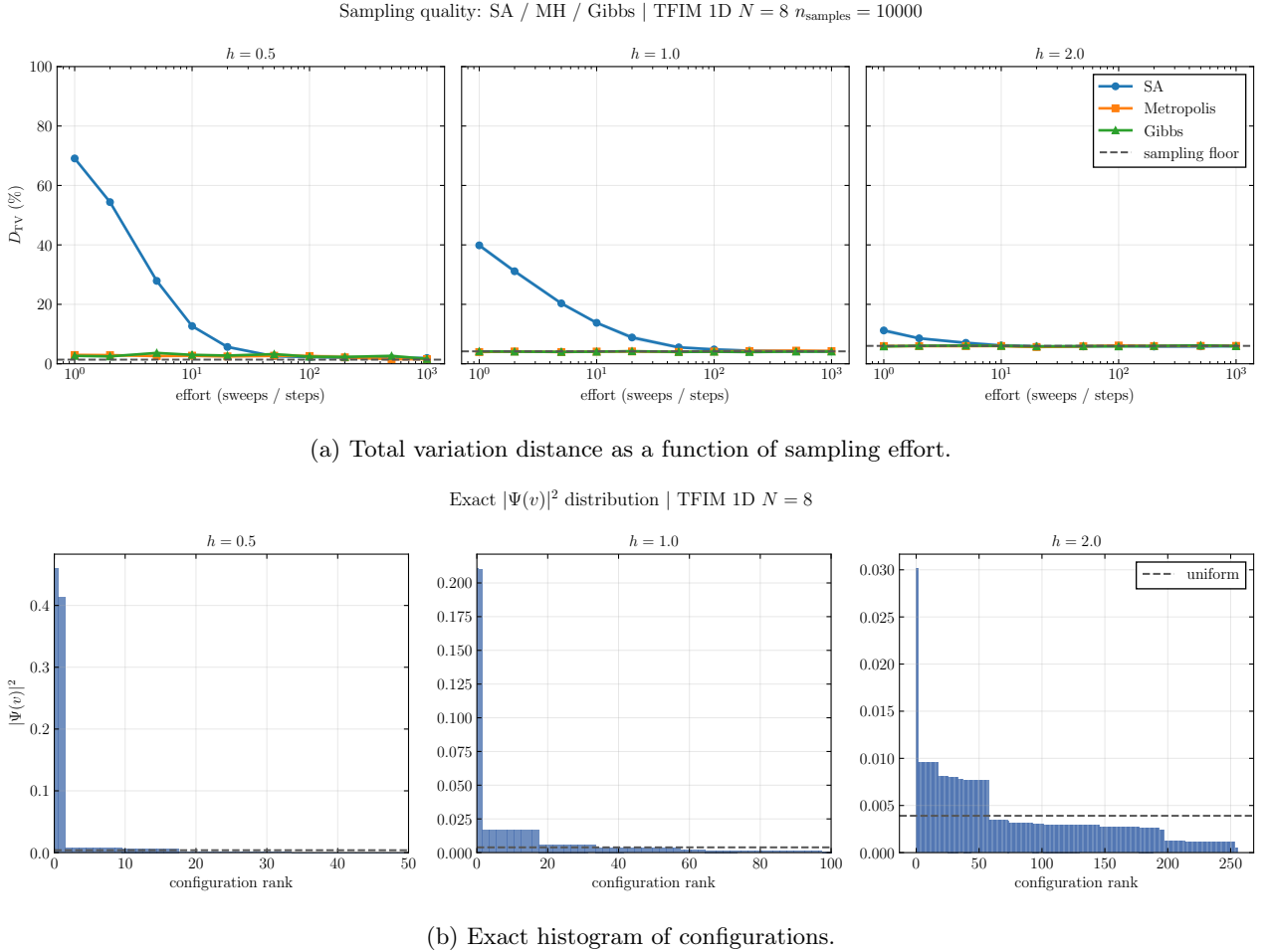


Figure 5: Sampling quality for different samplers for a trained RBM approximating the ground state of a TFIM with 8 spins, corresponding to a RBM with 8 visible and 8 hidden units. The sampling floor is the best possible total variation distance reachable with the given number of samples. Effort is measured in sweeps.

5.2 Sampling temperature & CEM

In this section, we evaluate the influence of temperature on different models and solvers. We focus on two quantities: the rescaling factor β_x and the effective temperature β_{eff} [26].

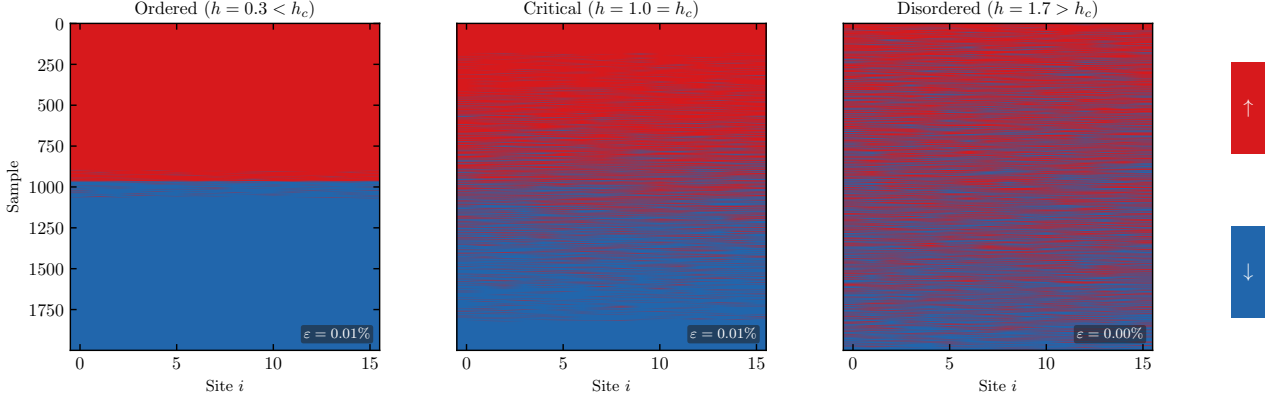


Figure 6: Spin configurations sampled from a trained RBM-VMC model for the one-dimensional TFIM at three values of the transverse field h . Each column shows 2000 spin configurations $\sigma \in \{-1, +1\}^N$ sampled from the learned wave function $|\Psi_\theta|^2$, sorted by total magnetisation. The model is a fully-connected RBM ($N = N_h = 16$) trained for 300 iterations of SR with 500 Metropolis-Hastings samples per step.

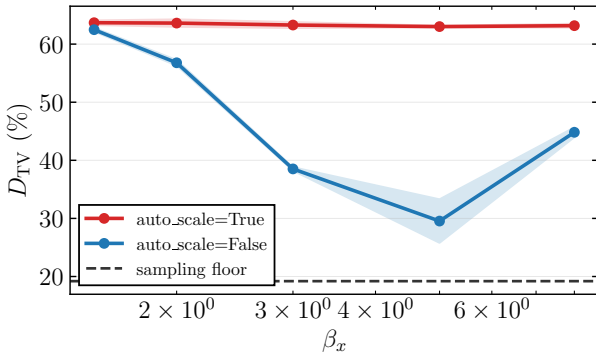
5.2.1 D-Wave QPU

The D-Wave has been found to reproduce a classical Boltzmann distribution[26], parametrized by a device- and problem-dependent temperature β_{hw} :

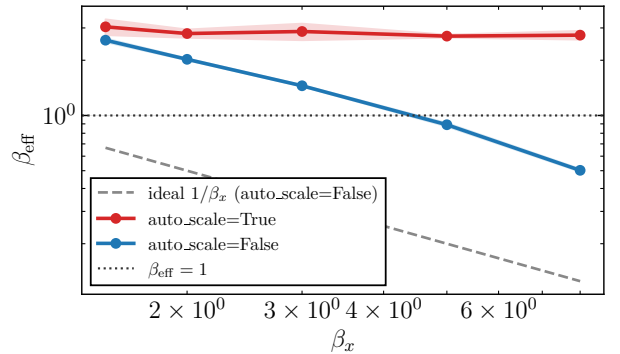
$$p(\mathbf{v}, \mathbf{h}) \propto e^{-\beta_{\text{hw}} E_{\text{in}}(\mathbf{v}, \mathbf{h})}. \quad (5.3)$$

Concretely, the RBM's own weights $\theta = \{\mathbf{a}, \mathbf{b}, \mathbf{W}\}$ are divided by β_x before being embedded on the QPU, i.e. $E_{\text{in}} = E_\theta(\mathbf{v}, \mathbf{h})/\beta_x$, so the hardware effectively samples the RBM's joint distribution at $\beta_{\text{eff}} = \beta_{\text{hw}}/\beta_x$. Therefore, all problems submitted to the QPU need to be rescaled by a factor β_x , which we estimate using Conditional Expectation Matching (CEM) (described in the next section).

A common pitfall when using the D-Wave QPU is its device dependent, limited parameter range for the h and J values. Using the API's default settings, these parameters get automatically rescaled to fully match the allowed range. While this is not problematic for a standard minimization problem, it does impact the temperature dependent sampling process, as any temperature adjust is without effect when autoscaling is turned on. We verify this on the `Advantage_system6` (Pegasus) chip, using a chain-free RBM embedded directly onto a native Pegasus biclique. Figure 7 sweeps β_x under both settings: with `auto_scale=True`, the total variation distance D_{TV} between the target and the sampled distribution stays mostly flat, as does the efficient temperature β_{eff} in Figure 7a. This clearly is not the case with `auto_scale=False`, upon reaching the optimal device sampling temperature, D_{TV} reduces to $\approx 30\%$. Therefore, all other D-Wave experiments in this work use `auto_scale=False`



(a) Total variation distance D_{TV} between the QPU samples and $|\Psi(v)|^2$.



(b) Effective inverse temperature β_{eff} , with the ideal $1/\beta_x$ line and the $\beta_{\text{eff}} = 1$ target.

Figure 7: Impact of `auto_scale` on β_x for a chain-free Pegasus-embedded TFIM RBM ($N = 8$, $h = 1$).

5.2.2 CEM

We have seen that correctly choosing the sampling temperature is crucial in obtaining meaningful distributions. CEM's original formulation [27] exploits that the hidden units of a Boltzmann machine are conditionally inde-

pendent given the visible units: for a visible configuration fixed to a condition vector $\mathbf{v} = \mathbf{r}$, the conditional expectation $\langle h_j \rangle = \tanh(\beta \Theta_j)$, $\Theta_j = b_j + \sum_i r_i W_{ij}$, is known in closed form for any β . To estimate the unknown β_{eff} , the sampler is queried repeatedly at the fixed condition $\mathbf{r}$ to obtain an empirical conditional average $\langle h_j \rangle_{\mathbf{r}, C}$, which is then matched against $\tanh(\beta \Theta_j)$ by minimizing the squared residual over β .

In this work we use a pooled variant of this matching step which reuses the joint samples $(\mathbf{v}_i, \mathbf{h}_i)_{i=1}^{N_s}$ the sampler already produces at each training iteration. We fit a single β_{eff} across the whole batch,

$$\beta_{\text{eff}} = \arg \min_{\beta} F(\beta), \quad F(\beta) = \sum_{i=1}^{N_s} \sum_{j=1}^{N_h} (h_{ij} - \tanh(\beta \cdot \Theta_j(\mathbf{v}_i)))^2, \quad (5.4)$$

with i being the sample index. h_{ij} is the observed hidden unit j in sample i , and $\Theta_j(\mathbf{v}_i) = b_j + \sum_l v_{i,l} W_{lj}$ with l the visible-unit index. This allows us to perform the CEM algorithm without additional sampling cost.

The matching step is visualized in Figure 8, where the left subfigure shows multiple candidate functions $\tanh(\beta \Theta)$ dependent on β that can be used to match the returned samples. The right subfigure describes the corresponding minimization, in which the squared loss function $F(\beta)$ is minimized over β (using scipy's `minimize_scalar` function).

For small sizes, the effective temperature β_{eff} can be calculated exactly using exact enumerations. To assess the quality of the temperature estimate given by CEM, we first need to calculate the actual β_{eff} numerically. This is done by first sampling from a temperature dependent sampler, follow by a minimization over β of the KL-divergence between the sampled probability distribution and the actual, β rescaled RBM distribution $\nu_{\beta}(\mathbf{v})$. This algorithm is performed over $N \in \{8, 12\}$, $h \in \{0.5, 1.0, 1.5, 2.0\}$, and $\beta_x \in \{0.5, 1.0, 1.5, 2.0\}$ with 5 seeds and three checkpoints during training (see Figure 9). Both subplots give the same picture: the β_{eff} calculated by CEM is very close to the exact solver temperature given by exact enumeration.

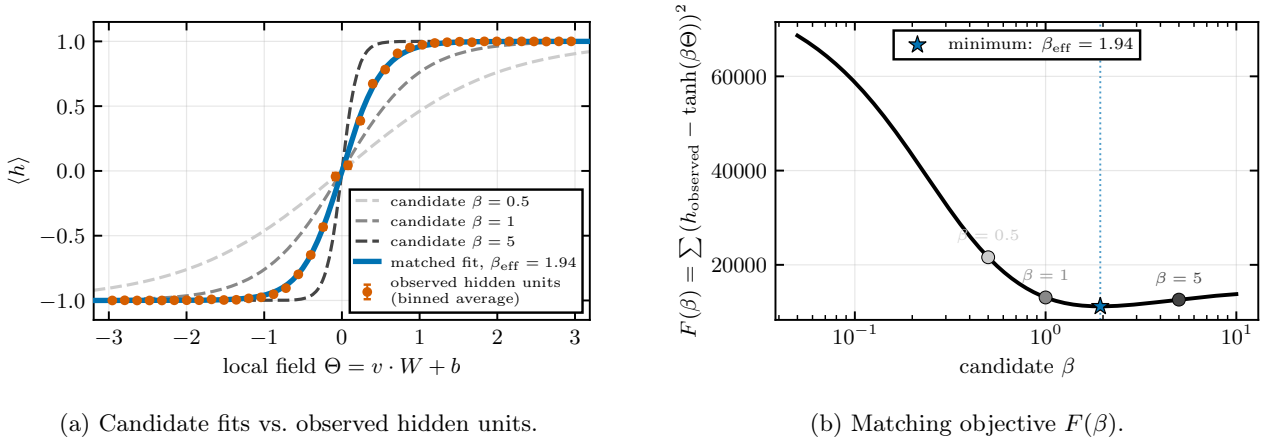


Figure 8: The CEM matching step for $N = 16$ spins with $h = 1.0$. (a) Several candidate $\tanh(\beta \Theta)$ curves against the empirically observed hidden-unit samples. The curve at $\beta_{\text{eff}} = 1.94$ approximates the data most closely and represents the best estimate of the sampler's temperature. (b) The squared loss function to be minimized.

5.2.3 Simulated Annealing

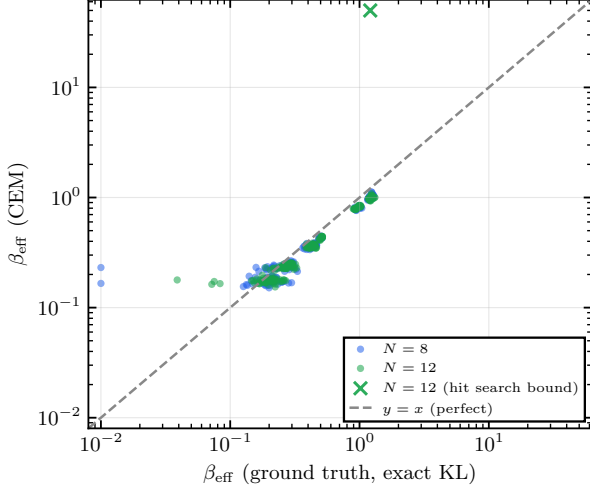
Simulated annealing is a Metropolis-Hastings chain held at a fixed final temperature β_x ; given enough sweeps it converges to the Boltzmann distribution at that temperature [17, 15]. Figure 10 demonstrates this experimentally, showing D_{TV} converge toward its sampling floor as the number of cooling sweeps increases.

5.3 Improving D-Wave Topology performance

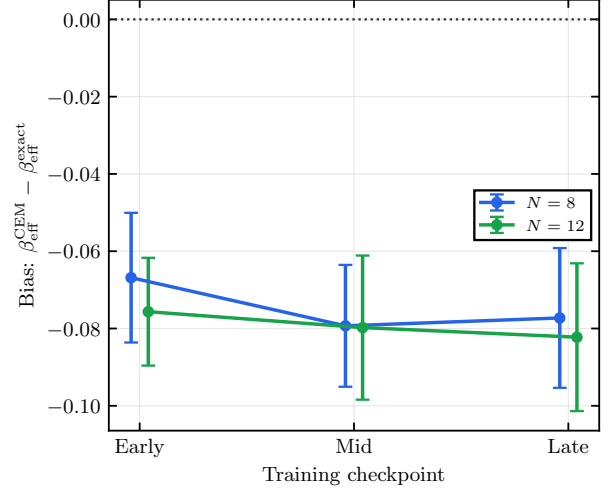
A crucial factor in the success of an annealing process is the *problem embedding* on the chip. Here, we assess two aspects related to embedding: First, introducing sparsity to fit the RBM natively on the chip, and, second, embedding the problem multiple times to save computing time.

5.3.1 Sparse embedding

As described in subsection 3.1, problems implementable on a quantum annealer often need to be fit to the QPU in an additional embedding step. We test whether a sparse, native RBM that fits directly on the QPU performs reasonably. The sparse RBM is created according to the following algorithm: First, construct a



(a) CEM vs. exact ground truth.



(b) Bias across training.

Figure 9: Validation of the CEM β_{eff} estimator against an exact estimate given by minimizing D_{KL} against the true visible marginal of the β -rescaled joint Boltzmann distribution (a) CEM’s estimate vs. the exact ground truth, the dashed lines gives the optimal values. (b) Bias of the CEM estimate relative to ground truth, averaged over h and β_x , at three checkpoints during training (10%/50%/100% of iterations). Error bars show 95% confidence intervals over seeds.

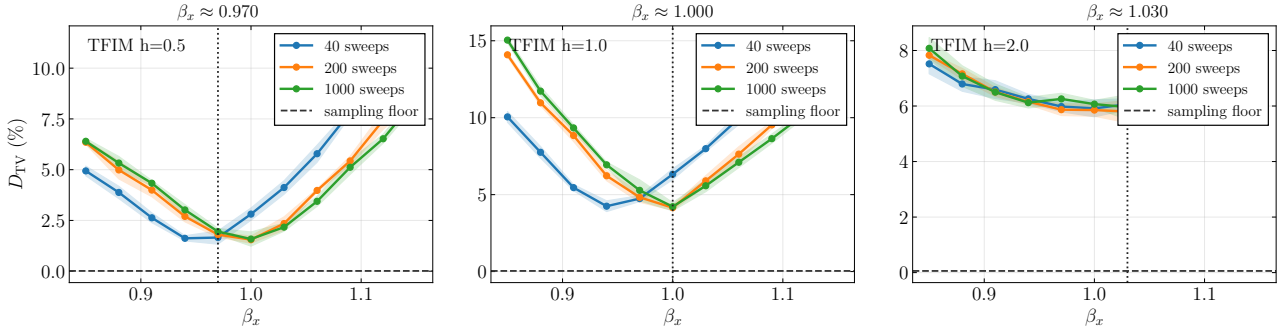


Figure 10: Total variation distance D_{TV} between SA’s empirical histogram and the exact distribution $|\Psi(v)|^2$, for a trained TFIM RBM with $N = 8$ visible spins, at three cooling-sweep counts. The horizontal dashed line marks the sampling floor; the vertical dotted line in each panel marks the β_x that minimises D_{TV} .

bipartite graph by keeping only edges that correspond to internal couplers on the D-Wave QPU. One set of the graph corresponds to visible RBM units, the other one to hidden units. Then, greedily pick qubits for the visible units, by maximizing the overlap between neighbor sets of visible units. After all visible units are chosen, create a minimum set-cover by picking the fewest hidden qubits such that every visible qubit has at least one hidden neighbor. The remaining hidden units are filled by best connected qubits. See also Figure 11 for a visual overview of the algorithm. We also look directly at the effect of sparsity on its own, keeping the number of hidden units equal to the number of visible units and only changing how many of the real couplers are removed from the connectivity mask. When removing couplers, we always remove one from the hidden unit with the highest remaining degree, so that no visible or hidden unit was ever left with zero connections. Figure 11 shows the resulting energy error as a function of sparsity, for three classical samplers (Metropolis-Hastings, simulated annealing, and persistent-chain Gibbs). In all three cases, error increases sharply as sparsity increases.

To establish the the best possible (theoretical) performance of a sparse RBM, we additionally trained the same four sparsity masks ($N = 16$, $h = 1$, zephyr, $\alpha = 1$) via exact enumeration of all 2^{16} visible configurations weighted by the exact $|\Psi(v)|^2$ probability, which removes all training bias from the experiment. We construct four artificial sparsity maps with the sparsity rate (the ratio of selected connection to the number of connections in a fully connected RBM) being $\in \{0.557, 0.682, 0.809, 0.877\}$. Making the RBM sparser than necessary allows us to estimate the impact of a native QPU Boltzmann machine for increasing sizes of N , that is, the total number of spins. As can be seen in the heatmap Figure 11d, the relative error not only depends on the chosen sparsity map, but also on the h value of the TFIM to be simulated. We conclude that a native Pegasus/Zephyr embedding is not suitable for efficient RBM sampling, for the following reasons. Even in the optimal case,

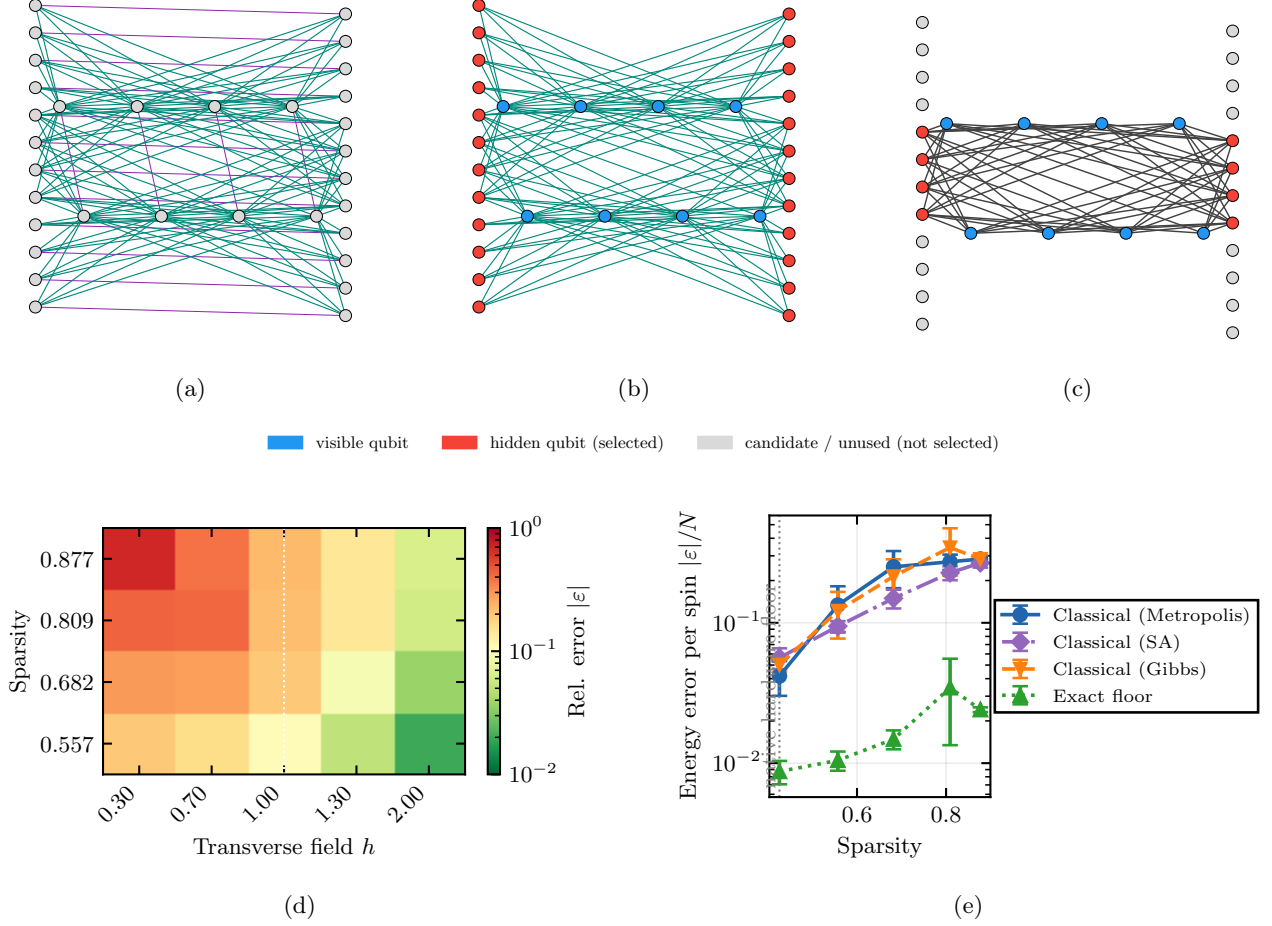


Figure 11: Construction and performance of a chain-free sparse RBM from a D-Wave Zephyr hardware graph. (a) Identify internal couplers (green) and external/odd couplers (purple) ($N = 8$ visible units shown). (b) Creation of bipartite graph by restricting to internal couplers, splitting into hidden unit candidates (red) and visible units (blue). (c) RBM's visible-hidden connectivity mask by selecting the hidden units which maximize connectivity while keeping a min-set-cover. Non-selected candidates (grey) are dropped. (d) Relative error $|\epsilon|/|E_{\text{exact}}|$ as a function of sparsity for different values of h , trained classically via Metropolis sampling. (e) Energy error per spin, $|\epsilon|/N$, at $h = 1$ across three classical samplers (Metropolis, simulated annealing, and Gibbs), against the exact-ansatz floor.

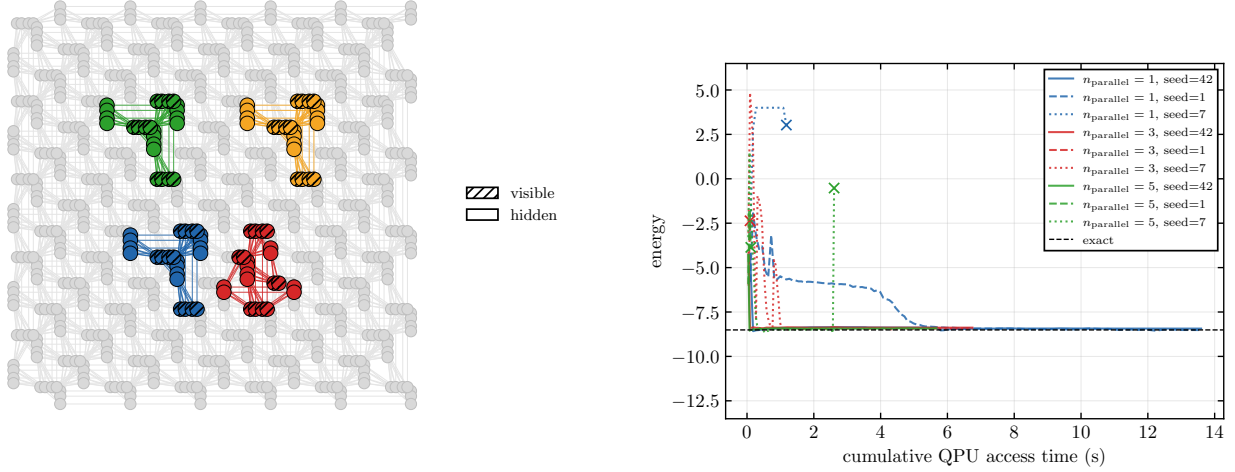
representability of the sparse ansatz is limited: the noise-free enumeration gives a lower bound that, in practice is hard to reach. Across three independent classical samplers (Metropolis-Hastings, simulated annealing, and persistent-chain Gibbs), trained under identical hyperparameters and iteration budgets, the per-spin error at every tested sparsity level sits 9.7–17.7 $\times$ above this floor, with no sampler avoiding the gap. Due to budget constraints, we tested the sparse models only on classical hardware.

5.3.2 Parallel Embeddings

To speed up calculation and save computing time, it is possible to embed multiple smaller, independent problems on the QPU [28]. For example, it is possible to gather 1000 independent samples using only 250 annealing runs, by embedding the RBM four times on the QPU. We experimentally validate this approach with 3 seeds per $n_{\text{parallel}} \in \{1, 3, 5\}$, as Figure 12 makes visible that training at this iteration budget is not reliable: 4 of the 9 runs diverge (crossed-off markers), including 2 of 3 seeds at $n_{\text{parallel}} = 5$. For the runs that do converge, Figure 12b indicates that parallel embedding can indeed be used to save QPU time. To find a parallel embedding, we use D-Wave's `dwave.system.ParallelEmbeddingComposite`.

5.4 Sign Problem in Quantum Monte Carlo

It is well known that capturing some quantum dynamics with non-trivial sign structure using a real-valued variational ansatz is not possible (*sign problem*) [29]. This does not pose a problem for the TFIM models, as it is a *stoquastic* model [30]. A stoquastic matrix is defined to have only real and non-positive off-diagonal elements.



(a) Four independent $K_{8,8}$ RBM embeddings ($n_{\text{parallel}} = 4$) placed simultaneously on the Pegasus chip. Colors distinguish copy 1-4; hatched markers denote visible units, solid markers denote hidden units.

(b) Energy vs. cumulative QPU access time for $n_{\text{parallel}} \in \{1, 3, 5\}$, all 3 seeds shown individually. $\times$ markers indicate a run diverged before reaching the plateau.

Figure 12: Parallel embedding on a D-Wave QPU: (a) physical embedding layout, (b) resulting training speedup.

It can easily be seen that the TFIM is stoquastic from its definition in Equation 2.1: the $\sigma_i^z \sigma_j^z$ terms only contribute diagonal elements, and the off-diagonal elements from the transverse field $\sigma_i^x \sigma_j^x$ all contribute the same factor $-h$. Some Heisenberg J_1 - J_2 models do not have this property and the corresponding RBM training converges far from the ground state (see Figure 14).

The reason for this is that the factors $e^{-\mathbf{a} \cdot \mathbf{v}/2}$ and $\cosh(\cdot)$ of the RBM ansatz defined in Equation 4.4 are strictly positive for real parameters and therefore the ansatz does not allow for the representation of negative amplitudes[21].

5.4.1 Marshall sign rule

Non-stoquasticity of the Heisenberg model To understand why the sign problem appears in the Heisenberg model and not in the TFIM, we need to assess the off-diagonal values of the system Hamiltonian. Writing a bond between sites i and j as $H_{ij} = J[\sigma_i^x \sigma_j^x + \sigma_i^y \sigma_j^y + \sigma_i^z \sigma_j^z]$, only the $\sigma_i^x \sigma_j^x + \sigma_i^y \sigma_j^y$ part is off-diagonal (it flips both spins of an antiparallel pair, e.g. $|\uparrow\downarrow\rangle \rightarrow |\downarrow\uparrow\rangle$), while $\sigma_i^z \sigma_j^z$ leaves the configuration unchanged. Using $\sigma^x |\uparrow\rangle = |\downarrow\rangle$, $\sigma^y |\uparrow\rangle = i |\downarrow\rangle$, and $\sigma^y |\downarrow\rangle = -i |\uparrow\rangle$, the two terms act on the antiparallel pair as

$$(\sigma_i^x \sigma_j^x + \sigma_i^y \sigma_j^y) |\uparrow\downarrow\rangle = |\downarrow\uparrow\rangle + (i)(-i) |\downarrow\uparrow\rangle = 2 |\downarrow\uparrow\rangle, \quad (5.5)$$

Consequently, every antiparallel bond contributes

$$\langle \downarrow\uparrow | H_{ij} | \uparrow\downarrow \rangle = 2J. \quad (5.6)$$

To restore stoquasticity, it is enough to simply multiply the off-diagonal elements E_{loc} contribution by -1 . This can be motivated by the Marshall sign rule [31], which is used to remove the non-trivial sign structure of the ground state wavefunction by basis transformation. Considering the following unitary transformation on bipartite lattices $A \cup B$ (two sublattices such that every nearest-neighbour bond connects a site in A to one in B , as on an alternating chain), written in the Pauli convention:

$$U = \prod_{i \in A} \sigma_i^z,$$

the ground state wavefunction can be written as

$$U|\psi_0\rangle = \sum_{\sigma} |c_{\sigma}| |\sigma\rangle,$$

with only positive amplitudes. This flip is exact for every J_1 bond, since bipartiteness guarantees each one connects A to B . A J_2 bond, however, connects two sites of the *same* sublattice so U leaves its sign unchanged. The correction is therefore exact only for $J_2 = 0$. It has however been shown that the Marshall correction

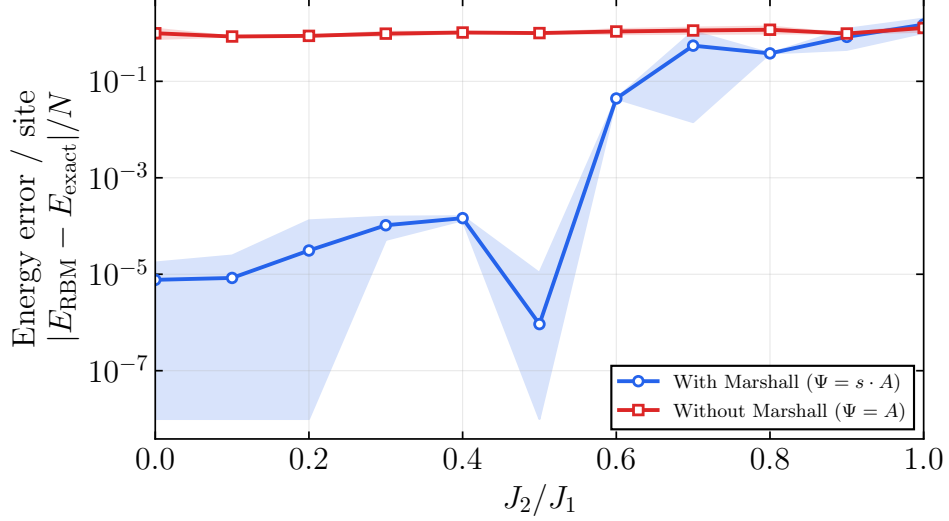


Figure 13: Energy error per site, $|E_{\text{RBM}} - E_{\text{exact}}|/N$, with and without the Marshall sign correction, against J_2/J_1 (mean $\pm$ std over seeds). Model trained using Metropolis-Hastings sampling, $N = 8$, 200 SR iterations and averaged over 3 seeds, swept over 11 points from 0 to 1.

can survive a certain degree of frustration (i.e. for J_2/J_1 up to a certain threshold)[32, 33], which we validate experimentally.

We validate this experimentally in Figure 13: applying the Marshall sign correction reduces the ground-state energy error by several orders of magnitude compared to the uncorrected ansatz, confirming that the correction works in practice. Consistent with [32]’s square-lattice result and [33]’s direct calculation for the 1D chain, this improvement persists over a range of frustration J_2/J_1 before degrading, verifying that the sign rule can empirically survive frustration beyond its proven regime of $J_2 = 0$. We did not test a phase-capable ansatz (e.g. a complex-valued RBM) and leave it to future work.

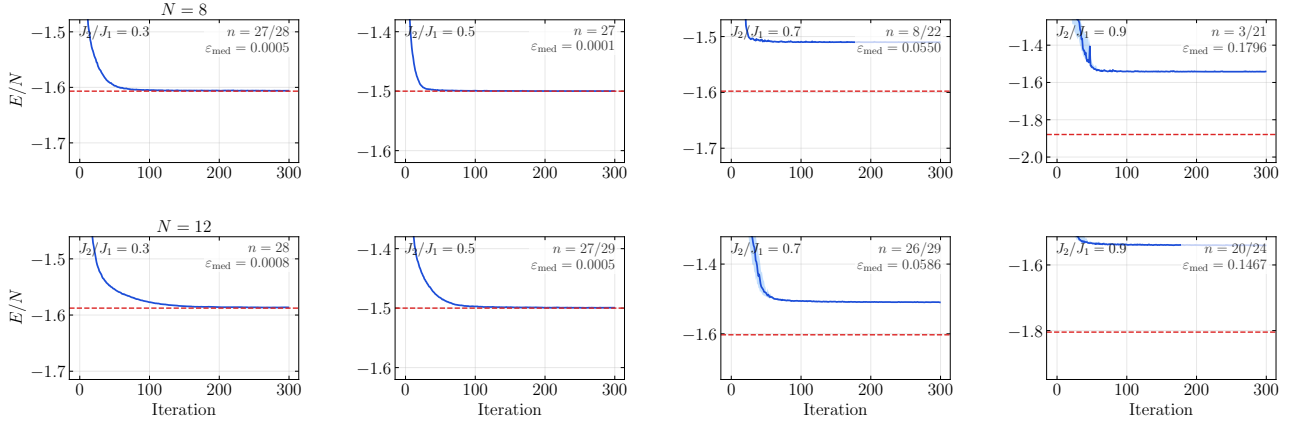


Figure 14: RBM convergence curves (E/N vs. SR iteration) for $N \in \{8, 12\}$ and $J_2/J_1 \in \{0.3, 0.5, 0.7, 0.9\}$ on the frustrated J_1 - J_2 Heisenberg chain, using Gibbs sampling. The solid line is the median over n independent evaluation seeds, the shaded band is the interquartile range. Where n is reported as a/b , b seeds were attempted and $b - a$ runs diverged under the selected configuration divergence is frequent in the strongly frustrated regime ($N = 8$, $J_2/J_1 \geq 0.7$). The red dashed line marks the exact ground-state energy E_{exact}/N .

5.5 Time-to- ϵ across solvers

5.5.1 Methodology

We present a hyperparameter matched solver comparison for the one-dimensional TFIM with $h = 0.5$ for N up to 64. Solvers are compared by pure sampling time only, that is without taking into account the classical training iteration overhead (i.e. SR calculation, which is identical across solvers). All runs are performed using the same hyperparameters (N hidden units (matching the visible count), learning rate 0.08, regularization 0.05, 200

samples per iteration, a 100-iteration budget, and 20 seeds per cell) and solver dependent internal parameters, which can be found in Table 1. For the D-Wave in particular, only the QPU’s own `qpu_access_time` (on-chip programming, anneal, and readout) is taken into account as returned by D-Wave’s Solver API (SAPI), one-time embedding and per-sampler network latency are excluded. For the classical solvers a separate one-time call is executed before training starts to trigger JAX’s just-in-time (JIT) compilation, so that cost is excluded from every reported iteration.

We report on time to ϵ (TTE)₉₉: following the same 99%-confidence convention already used for TTS in Equation 1.1, the expected time for a run to reach the solution to within a given energy tolerance ϵ with 99% confidence, computed from the validated (successful) fraction of seeds and their convergence time. The precise definition is given below.

A run is considered to converge when the rolling-window mean energy error per spin, $|\bar{E} - E_{\text{exact}}|/N$ averaged over the trailing 10 iterations, first drops below $\epsilon = 0.1$ and stays below it for 10 consecutive iterations. A run that does not satisfy this criterion within the 100-iteration budget is censored, i.e. does not validate. Pegasus and Zephyr are reported here with CEM correction only. For these series, the reported time also includes CEM’s β_{eff} estimation step which is added on top of the QPU’s own device-reported `qpu_access_time`.

Formally, with E_τ the training energy logged at SR iteration τ , window size $w = 10$, and iteration budget $T = 100$, the rolling-window mean and its error are

$$\bar{E}_t = \frac{1}{t - t_0 + 1} \sum_{\tau=t_0}^t E_\tau, \quad t_0 = \max(1, t - w + 1), \quad \delta_t = \frac{|\bar{E}_t - E_{\text{exact}}|}{N}. \quad (5.7)$$

The convergence iteration is the first sustained crossing for w iterations,

$$t^* = \min \left\{ t \leq T - w + 1 : \delta_s < \epsilon \quad \forall s \in \{t, \dots, t + w - 1\} \right\}, \quad (5.8)$$

undefined if no such t^* exists within budget. For a given solver and N , let p be the fraction of the 20 seeds for which t^* exists, and let

$$T_r = \text{median}_{\text{validated seeds}} \sum_{\tau=1}^{t^*} s_\tau \quad (5.9)$$

be the median wall-clock convergence time of those validated seeds (s_τ the time for one sampler call, so the sum is the cumulative sampling time up to and including iteration t^*). Analogously to TTS (Equation 1.1), a run that fails to validate is treated as a failure to be retried from scratch with a new seed, giving the 99%-confidence

$$\text{TTE}_{99} = \begin{cases} T_r, & p = 1, \\ T_r \frac{\ln(1 - 0.99)}{\ln(1 - p)}, & 0 < p < 1, \\ \text{undefined (censored)}, & p = 0. \end{cases} \quad (5.10)$$

5.5.2 Results analysis

Figure 15 (top) plots the median TTE₉₉ (with interquartile range) against system size $N \in \{8, 12, 16, 24, 32, 64\}$ on three panels: **(a)** classical MCMC samplers (Metropolis, Gibbs); **(b)** classical, physics-inspired heuristics (SA run locally on the GPU described in subsection A.2, and FPGA); and **(c)** the Pegasus and Zephyr QPUs with CEM correction. For budget reasons, we report QPU results at four sizes only: $N \in \{8, 16, 32, 64\}$.

FPGA is fastest through $N = 32$, where every series still validates 20/20 seeds and TTE₉₉ therefore equals T_r (no retries needed, $p = 1$). By $N = 64$, however, a handful of seeds no longer validate within budget for several series (FPGA 19/20, Zephyr(+CEM) 19/20, Metropolis 17/20, Gibbs 16/20, SA 18/20): FPGA’s TTE₉₉ (0.87 s) is now clearly higher than Pegasus(+CEM)’s. Gibbs stays essentially flat across N (with exponent $\propto N^{+0.65}$). We assume that this is due to a fixed one-time chain initialization (Table 1), which does not grow with N in this range (*persistent* contrastive divergence).

Figure 15 (bottom) reports the same validated-convergence cells in terms of GPU energy consumed instead, restricted to classical solvers. The D-Wave QPUs are omitted because no reliable energy consumption value could be established. FPGA energy is an assumed-constant-power estimate (45 W).

6 Conclusion

We assessed classical, quantum-annealing, and physics-inspired samplers within one framework, across the transverse-field Ising and Heisenberg models, using both time-to-solution and energy consumption. FPGA is the fastest and most energy-efficient of the metered classical solvers; QPU energy was not measured, so no classical-versus-quantum energy comparison is claimed. On time-to-solution, under the 99%-confidence-corrected TTE₉₉

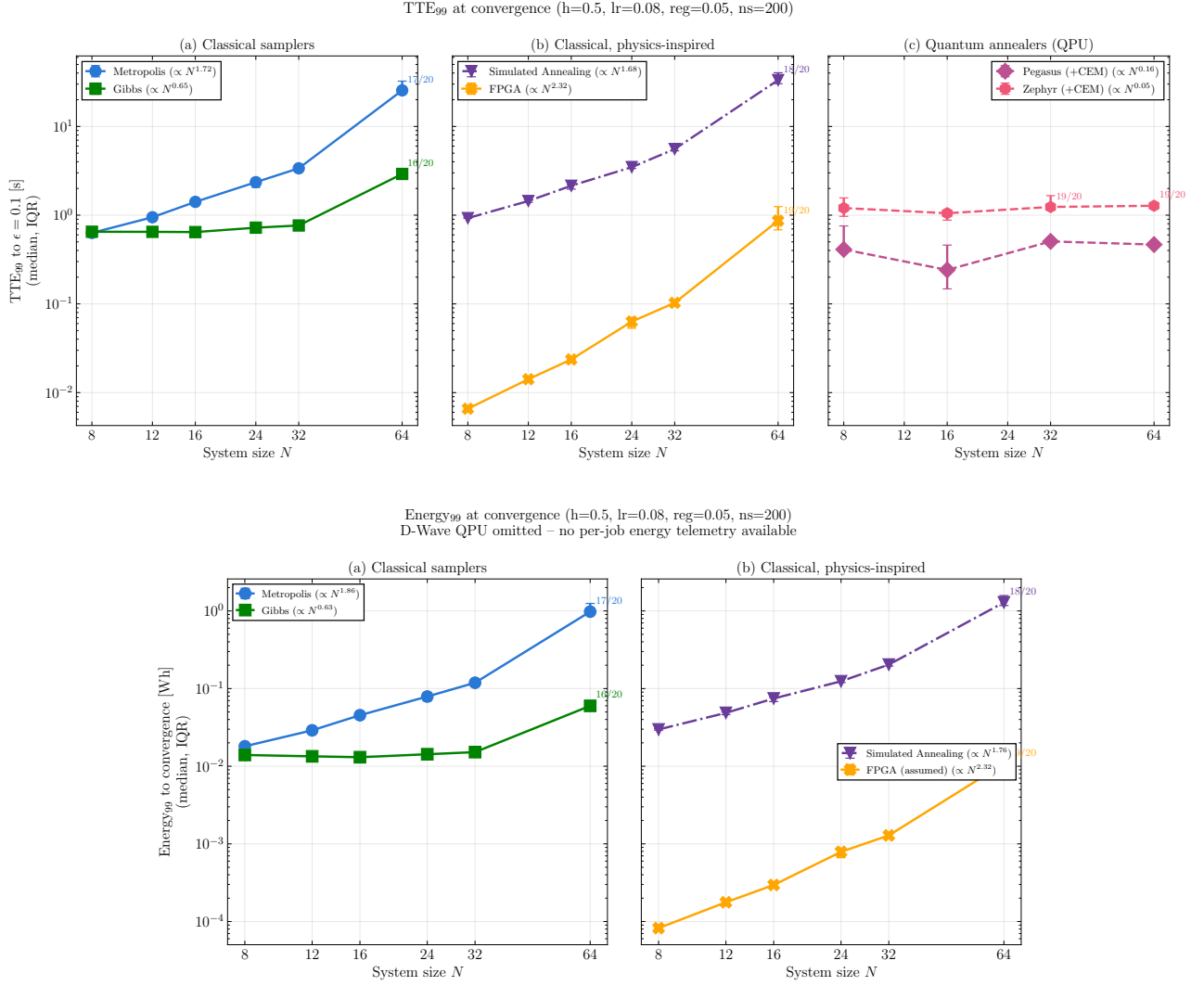


Figure 15: **(top)** Median TTE₉₉ (interquartile range) against system size N , under the protocol given in the text. Panels: **(a)** classical samplers, **(b)** classical physics-inspired heuristics, **(c)** quantum annealers (QPU, with CEM correction). Markers are annotated with the validated-over-total seed count whenever that fraction is below 20/20. Dotted lines show each series' fitted power law $\propto N^p$, with p given in the legend. **(bottom)** Median GPU energy₉₉ to the same validated-convergence cells, panels (a)–(b) only

metric, Pegasus(+CEM) and Zephyr(+CEM) both scale nearly flat with N and overtake FPGA by $N = 64$, reversing the near-tie a plain-median reading would suggest. This is not established *quantum advantage*, given the small number of system sizes tested and the excluded network/embedding overhead. We also assessed a native sparse embedding onto the annealer's hardware graph, which costs more accuracy than it saves in overhead. We investigated parallel embedding of multiple problem copies on the QPU and find it a promising direction for saving QPU time, though the seed count collected here is still small. We find that the variational ansatz used in this work cannot represent ground states with sign structure and analyze the impact of the Marshall sign rule, which restores accuracy for weakly frustrated Heisenberg chains, but accuracy degrades beyond moderate frustration.

Overall, FPGA remains the strongest sampler across the range of sizes tested here; the QPUs' late, criterion-sensitive overtake at $N = 64$ is a lead worth following up.

A Supplementary Material

A.1 JAX

(Quantum) variational algorithms do not only strain *quantum* resources, but also push *classical* computers to their limits. Notably, at each iteration, the SR process described in subsection 4.3 adapts all parameters of the weights and biases. We use several techniques in this project to speed up this process. JAX, implementing the *functional programming* paradigm, allows for the speed up of function calls and linear algebra calculation by offering features such as JIT compilation and XLA (accelerated linear algebra). We will describe the features of JAX in detail and give examples from our codebase.

A.1.1 Just-in-time compilation

JAX transformations require functions to be *functionally pure*. To summarize, a function is considered functionally pure, if its output depends only on the function’s input parameters, and all output is passed through the return variable. As an example, a function that involves `print()` statements or writes to global variables is not considered to be functionally pure, as its output does not exclusively pass through the `return` statement. JAX does not guarantee the proper behavior of impure functions. During JIT compilation, the function is transformed to primitives, which in JAX are defined to be "fundamental units of computation"[34]. As an example, the following function header is used to calculate the local energy (Equation 4.11):

```
@functools.partial(jax.jit, static_argnums=(4, 5))
def _local_energy_1d_jit(
    V: jax.Array, # (ns, N) -- spin configs
    W: jax.Array, # (N, M) -- RBM weights
    a: jax.Array, # (N,) -- visible biases
    b: jax.Array, # (M,) -- hidden biases
    h: float, # static
    N: int, # static
) -> jax.Array: # (ns,) local energies
    ...
```

h and N are passed as static parameters, meaning that they will be hard-coded in the compiled function after JIT. JAX compiles a kernel for each combination of static variables; as h and N never change during a training run, this means that the function is only compiled once.

A.1.2 Vectorized Map

`jax.vmap` takes a function written for a single input and returns a new function that maps it over a leading batch axis as a single vectorised kernel.

As a concrete example, consider `log_psi_fn` defined for a single spin configuration $\mathbf{v} \in \{-1, +1\}^N$. Evaluating it over all n_s sampled configurations reduces to:

```
log_p_V = jax.vmap(log_psi_fn)(V) # V: (ns, N) -> log_p_V: (ns,)
```

XLA compiles this as a single operation over the batch dimension rather than n_s independent kernel launches. The resulting operation can be compiled using JIT for faster execution:

```
batched_f = jax.jit(jax.vmap(log_psi_fn))
```

JAX also offers automatic differentiation, but we do not use it, since gradients are calculated analytically (see [8]).

A.2 Hardware and Software Configuration

All classical computation (classical and training loop) run locally via JAX on a single NVIDIA TITAN RTX GPU (24 GB, driver 560.35.03, CUDA 12.6). The FPGA backend runs on external hardware. D-Wave experiments use the `Advantage_system6` (Pegasus) and `Advantage2_system1` (Zephyr) solvers and were run between 2026-05-11 and 2026-08-10. Forward-anneal runs use a $20\mu\text{s}$ anneal time and `num_reads` equal to the sample count, with `auto_scale` disabled and with D-Wave’s default chain strength. Embeddings are cached per (`n_visible`, `solver`) using `minorminer`’s biclique embedder. FPGA runs use `Float32` precision, a 120 MHz core clock, and 1024 parallel replicas. Its reported time is an on-device cycle count divided by this clock frequency. All stochastic components are seeded per run directly and deterministically from the stated per-run integer seed: `jax.random.PRNGKey(seed)` is split to separately seed model initialization and the sampler before training starts.

Solver internal parameters. Each classical sampler in panels (a)-(b) has its own internal mixing/integration parameters, held fixed across every (N, seed) cell (see Table 1). Metropolis and Simulated Annealing restart their 200-chain ensemble from a uniform-random configuration at every SR iteration. Gibbs instead uses Persistent Contrastive Divergence (PCD), described in subsection 3.3.

Solver	Chain	Sweeps/iteration	Other parameters
Metropolis-Hastings	fresh	200 warmup + 1	single-spin-flip proposals
Gibbs (PCD)	persistent	10 (block)	one-time 200-sweep init
Simulated Annealing	fresh	200 warmup + 40 cooling	$T : 5.0 \rightarrow 1.0$ geometric schedule

Table 1: Internal parameters for the classical samplers of Figure 15. A sweep is N single-spin-flip (Metropolis/SA) or N -unit block (Gibbs) proposals.

A.3 Convergence trajectories across models, sizes, and solvers

Figure 16 shows the full training trajectories for the summary numbers in Figure 15 by plotting the median energy per SR iteration. The top block ($N = 8, 12, 16$) has four columns: the TFIM solved with classical MCMC (Metropolis, Gibbs, Simulated Annealing), the TFIM solved with the physics-inspired and hardware solvers (FPGA, D-Wave Pegasus+CEM, Zephyr+CEM), and the frustrated J_1 - J_2 Heisenberg chain at $J_2/J_1 = 0.3$ and 0.5 , each solved with Gibbs and Simulated Annealing. The bottom block ($N = 24, 32, 64$) only has the two TFIM columns, since we did not run the Heisenberg model past $N = 16$. All TFIM panels use the same matched hyperparameters as Figure 15 ($h = 0.5$, learning rate 0.08, regularization 0.05, 200 samples per iteration, 100 iterations, 20 seeds; see Table 1 for the solver-specific settings). The Heisenberg panels are not matched in the same way: each (N, method, J_2) cell instead uses whichever learning rate, regularization, and hidden-unit count we found was best using a prior hyperparameter search for that specific cell.

Code and data availability. The implementation, experiment scripts, and archived result caches underlying this report are available at <https://github.com/iitis/adiabatic-boltzmann>.

Appendix: convergence trajectories across models, sizes, and solvers
solid = median over seeds, band = IQR, dashed = exact ground-state energy

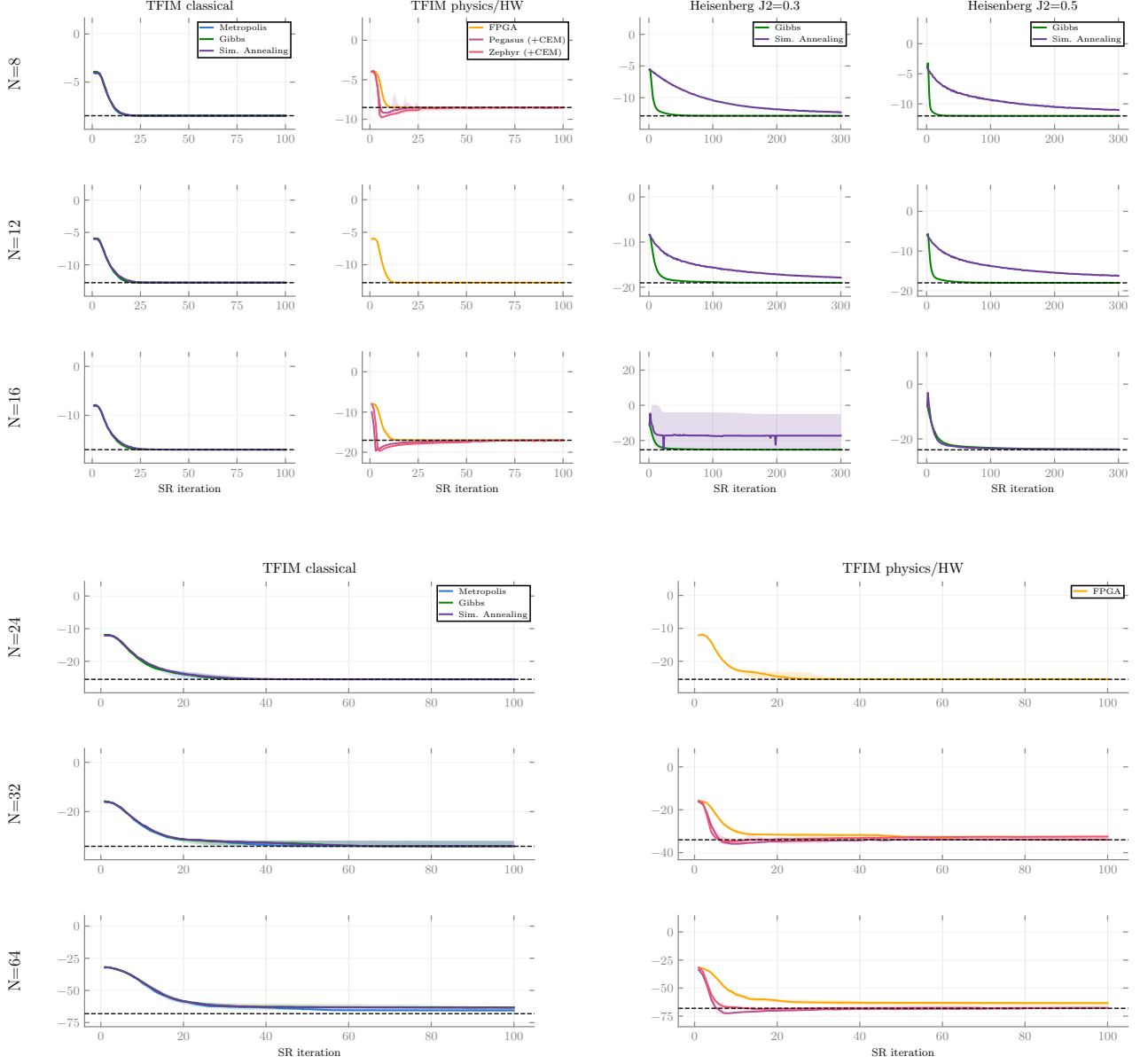


Figure 16: Median per-iteration energy (IQR band) across system size N , physical model, and solver, dashed line marks the exact ground-state energy. Top block: $N = 8, 12, 16$, all four columns. Bottom block: $N = 24, 32, 64$, TFIM columns only (Heisenberg archive stops at $N = 16$). TFIM columns use the Figure 15 matched cell; Heisenberg columns use each cell's own per-instance-tuned hyperparameters (see text).

References

- [1] Andrew D. King, Alberto Nocera, Marek M. Rams, et al. Beyond-classical computation in quantum simulation. *Science*, 388(6743):199–204, 2025. <https://doi.org/10.1126/science.ado6285>.
- [2] Philipp Hanussek, Jakub Pawłowski, Zakaria Mzaouali, and Bartłomiej Gardas. Quantum-inspired dynamical models on quantum and classical annealers. *Physical Review Applied*, 25(4):044055, April 2026. <https://doi.org/10.1103/rrf3-jm5m>.
- [3] Konrad Jałowiecki, Andrzej Więckowski, Piotr Gawron, and Bartłomiej Gardas. Parallel in time dynamics with quantum annealers. *Scientific Reports*, 10(1):13534, 2020. <https://doi.org/10.1038/s41598-020-70017-x>.
- [4] Dennis Willsch, Philipp Hanussek, Georg Hoever, Madita Willsch, Fengping Jin, Hans De Raedt, and Kristel Michielsen. The state of factoring on quantum computers. In *NIC Symposium 2025*, volume 52 of *NIC Series*, pages 239–250, 2025. Preprint: <https://arxiv.org/abs/2410.14397>.
- [5] Jingwen Ding, Giuseppe Spallitta, and Roberto Sebastiani. Effective prime factorization via quantum annealing by modular locally-structured embedding, 2023. <https://arxiv.org/abs/2310.17574>.
- [6] Vrinda Mehta, Hans De Raedt, Kristel Michielsen, and Fengping Jin. Understanding the physics of D-Wave annealers: From Schrödinger to Lindblad to Markovian dynamics. *Physical Review A*, 112(3):032616, September 2025. <https://doi.org/10.1103/3yk8-qn64>.
- [7] Jon Nelson, Marc Vuffray, Andrey Y. Lokhov, Tameem Albash, and Carleton Coffrin. High-quality thermal Gibbs sampling with quantum annealing hardware. *Physical Review Applied*, 17(4):044046, April 2022. <https://doi.org/10.1103/PhysRevApplied.17.044046>.
- [8] Bartłomiej Gardas, Marek M. Rams, and Jacek Dziarmaga. Quantum neural networks to simulate many-body quantum systems. *Physical Review B*, 98(18):184304, November 2018. <https://doi.org/10.1103/PhysRevB.98.184304>.
- [9] Alexander Wietek. Theory of quantum magnetism (lecture notes). https://www.pks.mpg.de/fileadmin/user_upload/MPIPKS/group_pages/SMC/quantum_magnetism_lecture_notes/lecture9.pdf. No publication date given in the source; [Accessed 29-06-2026].
- [10] W. Heisenberg. Zur theorie des ferromagnetismus. *Zeitschrift für Physik*, 49(9-10):619–636, 1928. <https://doi.org/10.1007/BF01328601>.
- [11] Elliott H. Lieb, Theodore Schultz, and Daniel C. Mattis. Two soluble models of an antiferromagnetic chain. *Annals of Physics*, 16(3):407–466, December 1961. [https://doi.org/10.1016/0003-4916\(61\)90115-4](https://doi.org/10.1016/0003-4916(61)90115-4).
- [12] Yan He and Hao Guo. The boundary effects of transverse field ising model. *Journal of Statistical Mechanics: Theory and Experiment*, 2017(9):093101, sep 2017. <https://doi.org/10.1088/1742-5468/aa85b0>.
- [13] M. Born and V. Fock. Beweis des adiabatensatzes. *Zeitschrift für Physik*, 51(3):165–180, 1928. <https://doi.org/10.1007/BF01343193>.
- [14] Edward Farhi, Jeffrey Goldstone, Sam Gutmann, and Michael Sipser. Quantum computation by adiabatic evolution. *arXiv preprint*, 2000. <https://arxiv.org/abs/quant-ph/0001106>.
- [15] Nicholas Metropolis, Arianna W. Rosenbluth, Marshall N. Rosenbluth, Augusta H. Teller, and Edward Teller. Equation of state calculations by fast computing machines. *The Journal of Chemical Physics*, 21(6):1087–1092, 1953. <https://doi.org/10.1063/1.1699114>.
- [16] W. K. Hastings. Monte Carlo sampling methods using Markov chains and their applications. *Biometrika*, 57(1):97–109, 1970. <https://doi.org/10.1093/biomet/57.1.97>.
- [17] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983. <https://doi.org/10.1126/science.220.4598.671>.
- [18] Xin-She Yang. Chapter 4 - simulated annealing. In *Nature-Inspired Optimization Algorithms*, pages 67–75. Elsevier, Oxford, 2014. <https://www.sciencedirect.com/science/article/pii/B978012416743800004X>.
- [19] Stuart Geman and Donald Geman. Stochastic relaxation, Gibbs distributions, and the Bayesian restoration of images. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-6(6):721–741, 1984. <https://doi.org/10.1109/TPAMI.1984.4767596>.

- [20] Francesco D’Angelo and Lucas Böttcher. Learning the ising model with generative neural networks. *Phys. Rev. Res.*, 2:023266, Jun 2020. <https://doi.org/10.1103/PhysRevResearch.2.023266>.
- [21] Giuseppe Carleo and Matthias Troyer. Solving the quantum many-body problem with artificial neural networks. *Science*, 355(6325):602–606, 2017. <https://doi.org/10.1126/science.aag2302>.
- [22] Nicolas Le Roux and Yoshua Bengio. Representational power of restricted boltzmann machines and deep belief networks. *Neural Comput.*, 20(6):1631–1649, June 2008. <https://doi.org/10.1162/neco.2008.04-07-510>.
- [23] Sandro Sorella. Green function Monte Carlo with stochastic reconfiguration. *Physical Review Letters*, 80(20):4558–4561, May 1998. <https://doi.org/10.1103/PhysRevLett.80.4558>.
- [24] Sandro Sorella. Generalized Lanczos algorithm for variational quantum Monte Carlo. *Physical Review B*, 64(2):024512, June 2001. <https://doi.org/10.1103/PhysRevB.64.024512>.
- [25] Magnus R. Hestenes and Eduard Stiefel. Methods of conjugate gradients for solving linear systems. *Journal of Research of the National Bureau of Standards*, 49(6):409–436, 1952. <https://doi.org/10.6028/jres.049.044>.
- [26] Marcello Benedetti, John Realpe-Gómez, Rupak Biswas, and Alejandro Perdomo-Ortiz. Estimation of effective temperatures in quantum annealers for sampling applications: A case study with possible applications in deep learning. *Physical Review A*, 94(2):022308, 2016. <https://doi.org/10.1103/PhysRevA.94.022308>.
- [27] Kentaro Kubo and Hayato Goto. Unlocking the power of Boltzmann machines by parallelizable sampler and efficient temperature estimation, 2025. <https://arxiv.org/abs/2512.02323>.
- [28] Elijah Pelofske, Georg Hahn, and Hristo N. Djidjev. Parallel quantum annealing. *Scientific Reports*, 12(1):4499, 2022. <https://doi.org/10.1038/s41598-022-08394-8>.
- [29] Matthias Troyer and Uwe-Jens Wiese. Computational complexity and fundamental limitations to fermionic quantum Monte Carlo simulations. *Physical Review Letters*, 94(17):170201, May 2005. <https://doi.org/10.1103/PhysRevLett.94.170201>.
- [30] Sergey Bravyi, David P. Divincenzo, Roberto Oliveira, and Barbara M. Terhal. The complexity of stoquastic local hamiltonian problems. *Quantum Info. Comput.*, 8(5):361–385, May 2008. <https://arxiv.org/abs/quant-ph/0606140>.
- [31] W. Marshall. Antiferromagnetism. *Proceedings of the Royal Society of London. Series A. Mathematical and Physical Sciences*, 232(1188):48–68, 1955. <https://doi.org/10.1098/rspa.1955.0200>.
- [32] J. Richter, N. B. Ivanov, and K. Retzlaff. On the violation of Marshall-Peierls sign rule in the frustrated J_1 - J_2 Heisenberg antiferromagnet. *Europhysics Letters*, 25(7):545–550, 1994. <https://doi.org/10.1209/0295-5075/25/7/012>.
- [33] Andreas Voigt and Johannes Richter. Marshall-Peierls sign rule in frustrated Heisenberg chains. *Acta Physica Polonica A*, 97(5):979, 2000. <https://arxiv.org/abs/cond-mat/0003207>.
- [34] JAX Developers. Glossary of terms — JAX documentation — docs.jax.dev. <https://docs.jax.dev/en/latest/glossary.html>, 2026. [Accessed 30-06-2026].